

# TRÌNH BIÊN DỊCH

## Chương 2. Phân tích cú pháp

Phạm Nguyễn Khang  
BM. Khoa học máy tính

[pnkhang@cit.ctu.edu.vn](mailto:pnkhang@cit.ctu.edu.vn)

CẦN THƠ, 12 - 2012

```
#include ...  
int main() {  
    ...  
}
```



Phân tích từ vựng

Phân tích cú pháp

Phân tích ngữ nghĩa

Sinh mã trung gian

Tối ưu mã trung gian

Sinh mã

Tối ưu

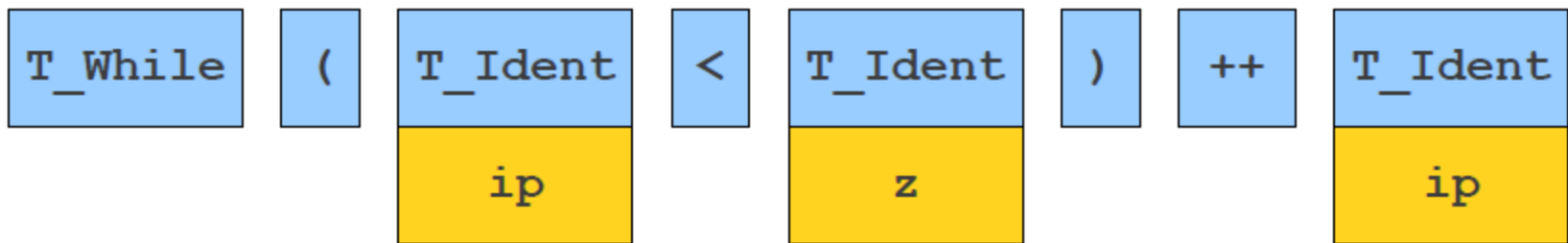


```
0001001000  
1011011100  
1100101111  
1000011111
```

```
while (ip < z)  
    ++ip;
```

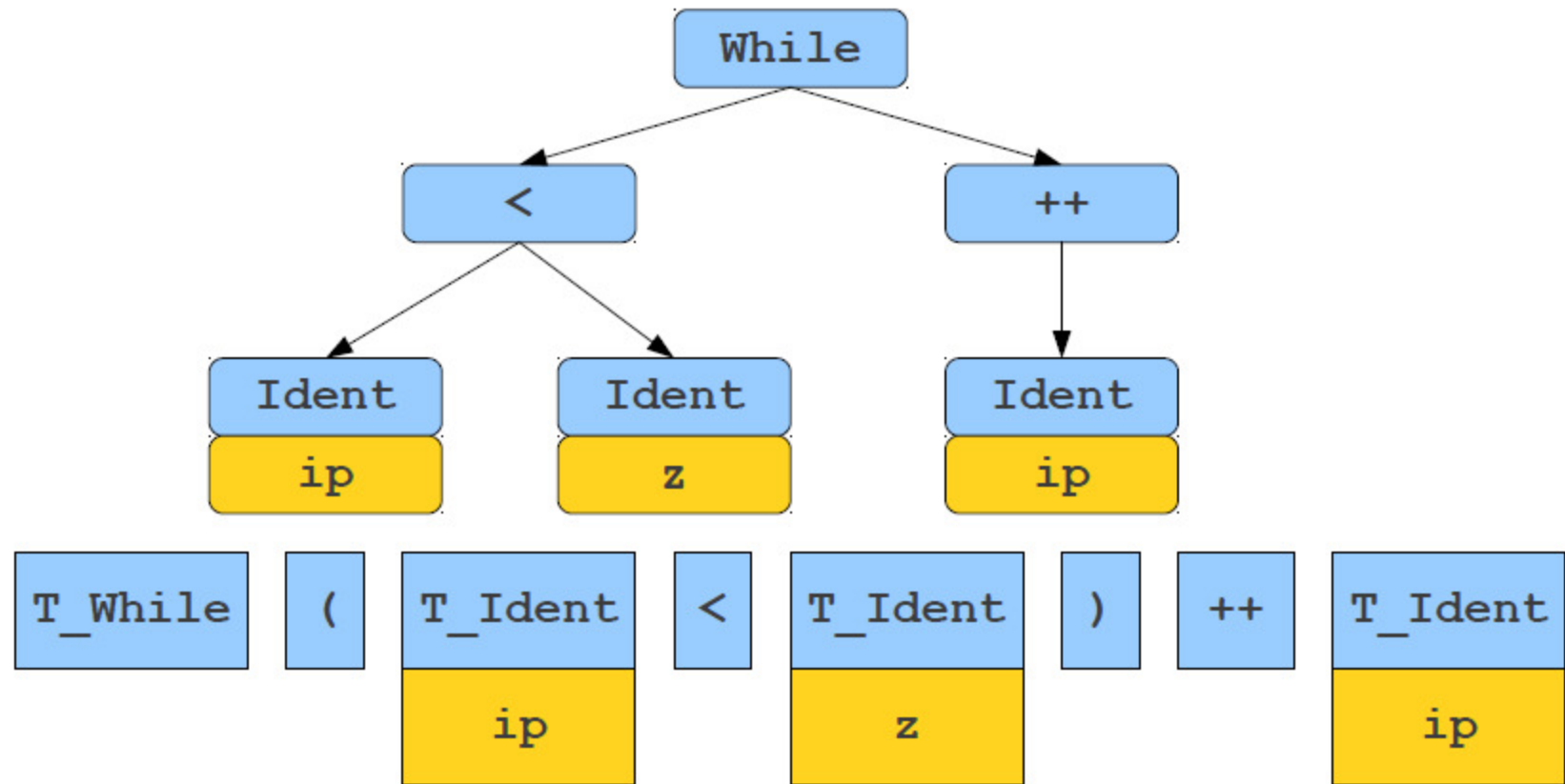
|   |   |   |   |   |  |   |   |   |  |   |  |   |   |    |    |   |   |   |   |   |
|---|---|---|---|---|--|---|---|---|--|---|--|---|---|----|----|---|---|---|---|---|
| w | h | i | l | e |  | ( | i | p |  | < |  | z | ) | \n | \t | + | + | i | p | ; |
|---|---|---|---|---|--|---|---|---|--|---|--|---|---|----|----|---|---|---|---|---|

```
while (ip < z)
    ++ip;
```



|   |   |   |   |   |  |   |   |   |  |   |  |   |   |    |    |   |   |   |   |   |
|---|---|---|---|---|--|---|---|---|--|---|--|---|---|----|----|---|---|---|---|---|
| w | h | i | l | e |  | ( | i | p |  | < |  | z | ) | \n | \t | + | + | i | p | ; |
|---|---|---|---|---|--|---|---|---|--|---|--|---|---|----|----|---|---|---|---|---|

```
while (ip < z)
    ++ip;
```



|   |   |   |   |   |  |   |   |   |  |   |  |   |   |    |    |   |   |   |   |   |
|---|---|---|---|---|--|---|---|---|--|---|--|---|---|----|----|---|---|---|---|---|
| w | h | i | l | e |  | ( | i | p |  | < |  | z | ) | \n | \t | + | + | i | p | ; |
|---|---|---|---|---|--|---|---|---|--|---|--|---|---|----|----|---|---|---|---|---|

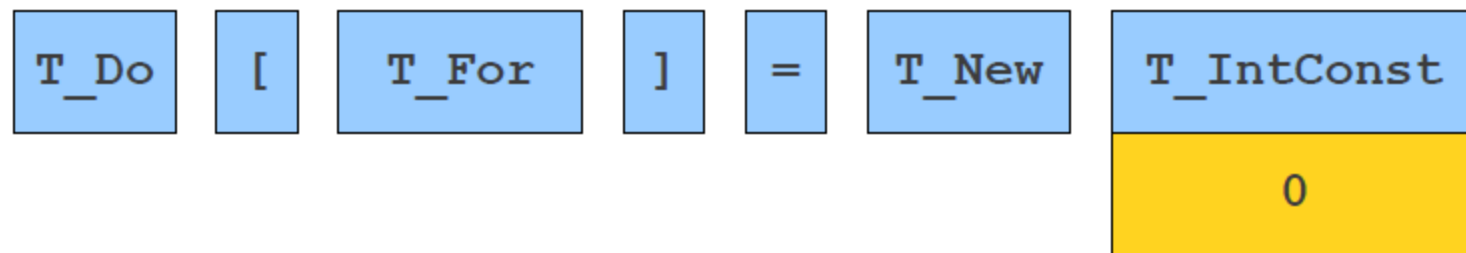
```
while (ip < z)
    ++ip;
```

```
do[for] = new 0;
```

|   |   |   |   |   |   |   |  |   |  |   |   |   |  |   |   |
|---|---|---|---|---|---|---|--|---|--|---|---|---|--|---|---|
| d | o | [ | f | o | r | ] |  | = |  | n | e | w |  | 0 | ; |
|---|---|---|---|---|---|---|--|---|--|---|---|---|--|---|---|

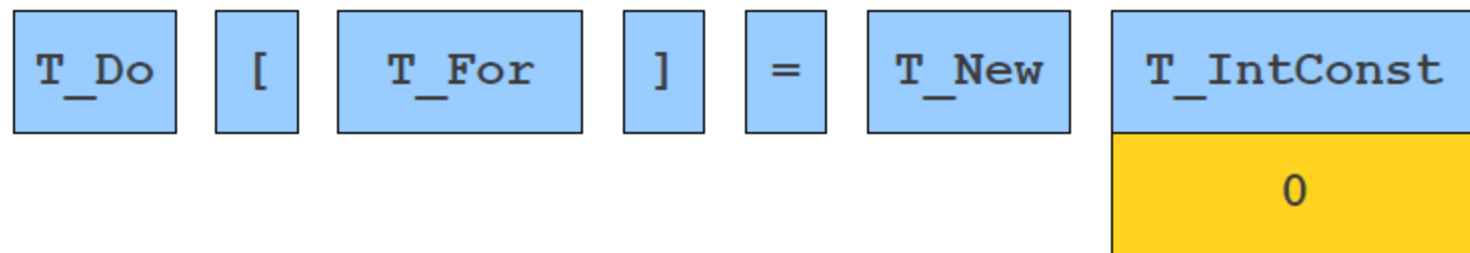
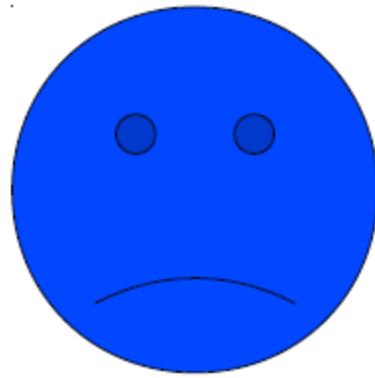
do[for] = new 0;





|   |   |   |   |   |   |   |  |   |  |   |   |   |  |   |   |
|---|---|---|---|---|---|---|--|---|--|---|---|---|--|---|---|
| d | o | [ | f | o | r | ] |  | = |  | n | e | w |  | 0 | ; |
|---|---|---|---|---|---|---|--|---|--|---|---|---|--|---|---|

`do[for] = new 0;`



|   |   |   |   |   |   |   |  |   |  |   |   |   |  |   |   |
|---|---|---|---|---|---|---|--|---|--|---|---|---|--|---|---|
| d | o | [ | f | o | r | ] |  | = |  | n | e | w |  | 0 | ; |
|---|---|---|---|---|---|---|--|---|--|---|---|---|--|---|---|

do[for] = new 0;

# Phân tích cú pháp là gì ?

- Sau khi phân tích từ vựng ta thu được **chuỗi các tokens**.
- Trong phân tích cú pháp (**syntax analysis** hay **parsing**), ta muốn diễn giải ý nghĩa của chuỗi các tokens này.
- Mục tiêu:
  - Dựng lại (recover) **cấu trúc** mà chuỗi các tokens này mô tả.
  - **Báo lỗi** nếu chuỗi các tokens này sai văn phạm

# Nội dung

- Tuần này: Phân tích cú pháp – các khái niệm
  - Văn phạm phi ngữ cảnh (Context-Free Grammars)
  - Dẫn xuất (Derivations)
  - Cây phân tích cú pháp trừu tượng và cụ thể (Concrete and Abstract Syntax Trees)
  - Mơ hồ (Ambiguity)
- Các tuần kế tiếp: Các giải thuật phân tích cú pháp
  - Phân tích từ trên xuống (Top-Down Parsing)
  - Phân tích từ dưới lên (Bottom-Up Parsing)

# Ngôn ngữ hình thức

- **Bảng chữ cái (alphabet)**, kí hiệu  $\Sigma$ :
  - Là một tập các ký hiệu (symbols). Mỗi kí hiệu được xem như một chữ cái hay ký tự.
  - Ví dụ: bảng chữ cái latin: A, B, C, ..., X, Y, Z
- Một **ngôn ngữ trên  $\Sigma$**  là một tập các chuỗi được tạo nên từ các ký hiệu trong  $\Sigma$ .
- Khi phân tích từ vựng, bảng chữ cái của chúng ta là bảng mã ASCII hoặc Unicode. Và ta sinh ra các tokens.
- Khi phân tích cú pháp, bảng chữ cái của chúng ta là tập các tokens được sinh ra từ bộ phân tích từ vựng.

# Hạn chế của ngôn ngữ chính quy

- Khi phân tích từ vựng, ta sử dụng các biểu thức chính quy để định nghĩa các token.
- Tuy nhiên, biểu thức chính quy thường **không đủ mạnh** để định nghĩa các ngôn ngữ lập trình. Ví dụ:
  - Không thể nào định nghĩa được một biểu thức chính quy mô tả chuỗi các dấu ngoặc cân bằng.
  - Không thể nào định nghĩa được một biểu thức chính quy mô tả các cấu trúc lồng nhau.
- Ta cần một ngôn ngữ hình thức mạnh hơn

# Văn phạm phi ngữ cảnh

- **Văn phạm phi ngữ cảnh** (context-free grammar viết tắt là CFG) là một phương pháp hình thức để định nghĩa các ngôn ngữ
- Có thể dùng CFG để định nghĩa các ngôn ngữ phi ngữ cảnh (context-free languages). Ngôn ngữ phi ngữ cảnh bao trùm luôn cả ngôn ngữ chính quy.
  - Có thể dùng CFG để định nghĩa ngôn ngữ chính quy
  - Không thể dùng biểu thức chính quy để định nghĩa ngôn ngữ phi ngữ cảnh.
- Ta sẽ giải thích CFG thông qua các ví dụ

# Biểu thức số học

- Giả sử ta muốn mô tả các biểu thức số học hợp lệ với các phép toán cộng, trừ, nhân và chia.
- Ta có thể định nghĩa bằng một văn phạm phi ngữ cảnh như sau:

- |                                                     |                                                            |
|-----------------------------------------------------|------------------------------------------------------------|
| • <b><math>E \rightarrow \text{int}</math></b>      | <b><math>E</math></b>                                      |
| • <b><math>E \rightarrow E \text{ Op } E</math></b> | <b><math>E \text{ Op } E</math></b>                        |
| • <b><math>E \rightarrow (E)</math></b>             | <b><math>E \text{ Op } (E)</math></b>                      |
| • <b><math>\text{Op} \rightarrow +</math></b>       | <b><math>E \text{ Op } (E \text{ Op } E)</math></b>        |
| • <b><math>\text{Op} \rightarrow -</math></b>       | <b><math>E * (E \text{ Op } E)</math></b>                  |
| • <b><math>\text{Op} \rightarrow *</math></b>       | <b><math>\text{int} * (E \text{ Op } E)</math></b>         |
| • <b><math>\text{Op} \rightarrow /</math></b>       | <b><math>\text{int} * (\text{int Op } E)</math></b>        |
| •                                                   | <b><math>\text{int} * (\text{int Op int})</math></b>       |
| •                                                   | <b><math>\text{int} * (\text{int} + \text{int})</math></b> |



# Văn phạm phi ngữ cảnh

- Một cách hình thức, một văn phạm phi ngữ cảnh là một bộ tứ:
  - Một tập các kí hiệu **chưa kết thúc** (còn gọi là biến)
  - Một tập các **ký hiệu kết thúc**,
  - Một tập các **luật sinh** mô tả cách biến đổi một ký hiệu chưa kết thúc thành một chuỗi các ký hiệu kết thúc và chưa kết thúc
  - Một **ký hiệu bắt đầu** (start symbol) bắt đầu quá trình dẫn xuất

$E \rightarrow \text{int}$

$E \rightarrow E \text{ Op } E$

$E \rightarrow (E)$

$\text{Op} \rightarrow +$

$\text{Op} \rightarrow -$

$\text{Op} \rightarrow *$

$\text{Op} \rightarrow /$

# Văn phạm phi ngữ cảnh (viết tắt)

- $E \rightarrow \text{int}$
- $E \rightarrow E \text{ Op } E$
- $E \rightarrow (E)$
- $\text{Op} \rightarrow +$
- $\text{Op} \rightarrow -$
- $\text{Op} \rightarrow *$
- $\text{Op} \rightarrow /$
- $E \rightarrow \text{int} \mid E \text{ Op } E \mid (E)$
- $\text{Op} \rightarrow + \mid - \mid * \mid /$

# Văn phạm phi ngữ cảnh (viết tắt)

- Không giống như biểu thức chính quy, ta không thể sử dụng  $*$ ,  $|$  và các dấu ngoặc
- Ví dụ:
  - $S \rightarrow a^*b$
- Sẽ được biểu diễn thành:
  - $S \rightarrow Ab$
  - $A \rightarrow Aa \mid \epsilon$

# Văn phạm phi ngữ cảnh (viết tắt)

- Không giống như biểu thức chính quy, ta không thể sử dụng  $*$ ,  $|$  và các dấu ngoặc
- Ví dụ:
  - $S \rightarrow a(b | c^*)$
- Sẽ được biểu diễn thành:
  - $S \rightarrow aX$
  - $A \rightarrow (b | c^*) \rightarrow X \rightarrow b | c^* \rightarrow X \rightarrow b | C$
  - $C \rightarrow Cc | \epsilon$

# CFG cho ngôn ngữ lập trình

BLOCK  $\rightarrow$  STMT  
          | { STMTS }  
STMTS  $\rightarrow \epsilon$   
          | STMT STMTS  
STMT  $\rightarrow$  EXPR;  
          | if (EXPR) BLOCK  
          | while (EXPR) BLOCK  
          | do BLOCK while (EXPR);  
          | BLOCK  
          | ...  
EXPR  $\rightarrow$  identifier  
          | constant  
          | EXPR + EXPR  
          | EXPR - EXPR  
          | EXPR \* EXPR  
          | ...

# Quy ước ký pháp

- Chữ hoa đầu bảng chữ cái biểu diễn kí tự chưa kết thúc. Ví dụ: **A, B, C, D**
- Chữ thường cuối bảng chữ cái biểu diễn các kí hiệu kết thúc. Ví dụ: **t, u, v, w**
- Chữ hy lạp thường biểu diễn chuỗi các ký tự kết thúc và chưa kết thúc. Ví dụ:  **$\alpha$ ,  $\gamma$ ,  $\omega$**

## Ví dụ

- Ta có thể viết một luật sinh như sau:
- **A**  $\rightarrow \omega$
- Ta có thể viết một chuỗi gồm một ký tự chưa kết thúc và một ký tự kết thúc như sau
- **A****t**
- Ta có thể viết một luật sinh bất kỳ chứa các ký tự kết thúc và chưa kết thúc như sau
- **B**  $\rightarrow a**A**tw$

# Dẫn xuất

$\Rightarrow E$   
 $\Rightarrow E \text{ Op } E$   
 $\Rightarrow E \text{ Op } (E)$   
 $\Rightarrow E \text{ Op } (E \text{ Op } E)$   
 $\Rightarrow E * (E \text{ Op } E)$   
 $\Rightarrow \text{int} * (E \text{ Op } E)$   
 $\Rightarrow \text{int} * (\text{int Op } E)$   
 $\Rightarrow \text{int} * (\text{int Op int})$   
 $\Rightarrow \text{int} * (\text{int} + \text{int})$

- Chuỗi các bước này gọi là một **dẫn xuất** (derivation).
- Chuỗi  $\alpha A \omega$  sinh ra chuỗi  $\alpha \gamma \omega$  nếu và chỉ nếu  $A \rightarrow \gamma$  là một luật sinh.
- Nếu  $\alpha$  sinh ra  $\beta$ , ta viết  $\alpha \Rightarrow \beta$ .
- Ta nói rằng  $\alpha$  dẫn xuất ra (derives)  $\beta$  nếu và chỉ nếu tồn tại dãy:
  - $\alpha \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \beta$
  - Nếu  $\alpha$  dẫn xuất ra  $\beta$ , ta viết  $\alpha \Rightarrow^* \beta$ .



# Dẫn xuất trái nhất

**BLOCK** → **STMT**  
| { **STMTS** }

**STMTS** →  $\epsilon$   
| **STMT STMTS**

**STMT** → **EXPR;**  
| **if** (**EXPR**) **BLOCK**  
| **while** (**EXPR**) **BLOCK**  
| **do** **BLOCK** **while** (**EXPR**);  
| **BLOCK**  
| ...

**EXPR** → **identifier**  
| **constant**  
| **EXPR + EXPR**  
| **EXPR - EXPR**  
| **EXPR \* EXPR**  
| **EXPR = EXPR**  
| ...

**STMTS**

⇒ **STMT STMTS**

⇒ **EXPR; STMTS**

⇒ **EXPR = EXPR; STMTS**

⇒ **id = EXPR; STMTS**

⇒ **id = EXPR + EXPR; STMTS**

⇒ **id = id + EXPR; STMTS**

⇒ **id = id + constant; STMTS**

⇒ **id = id + constant;**

# Dẫn xuất trái nhất

- **Dẫn xuất trái nhất** là dẫn xuất trong đó ở mỗi bước ta triển khai ký tự chưa kết thúc trái nhất.
- **Dẫn xuất phải nhất** là dẫn xuất trong đó ở mỗi bước ta triển khai ký tự chưa kết thúc phải nhất.
- Đây là một vấn đề quan trọng khi phân tích cú pháp.

## 2 loại dẫn xuất

$\Rightarrow E$

$\Rightarrow E \text{ Op } E$

$\Rightarrow \text{int Op } E$

$\Rightarrow \text{int} * E$

$\Rightarrow \text{int} * (E)$

$\Rightarrow \text{int} * (E \text{ Op } E)$

$\Rightarrow \text{int} * (\text{int Op } E)$

$\Rightarrow \text{int} * (\text{int} + E)$

$\Rightarrow \text{int} * (\text{int} + \text{int})$

$\Rightarrow E$

$\Rightarrow E \text{ Op } E$

$\Rightarrow E \text{ Op } (E)$

$\Rightarrow E \text{ Op } (E \text{ Op } E)$

$\Rightarrow E \text{ Op } (E \text{ Op } \text{int})$

$\Rightarrow E \text{ Op } (E + \text{int})$

$\Rightarrow E \text{ Op } (\text{int} + \text{int})$

$\Rightarrow E * (\text{int} + \text{int})$

$\Rightarrow \text{int} * (\text{int} + \text{int})$

# Dẫn xuất

- Một dẫn xuất mã hóa 2 thông tin:
  - Các luật sinh nào được sử dụng để sinh ra chuỗi kết quả từ ký tự bắt đầu ?
  - Thứ tự của các luật được sử dụng là gì ?
- Nhiều dẫn xuất có thể sử dụng các luật sinh giống nhau, nhưng thứ tự sử dụng khác nhau.

# Cây phân tích cú pháp

E

E

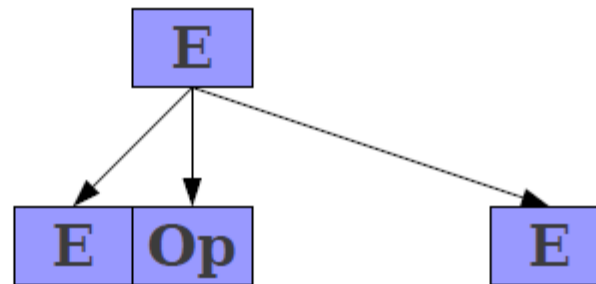
# Cây phân tích cú pháp

E

**E**  
 $\Rightarrow$  **E Op E**

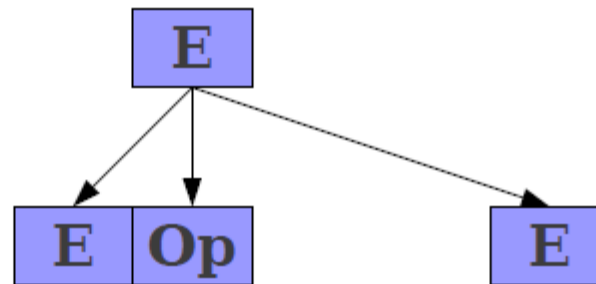
# Cây phân tích cú pháp

**E**  
 $\Rightarrow$  **E Op E**



# Cây phân tích cú pháp

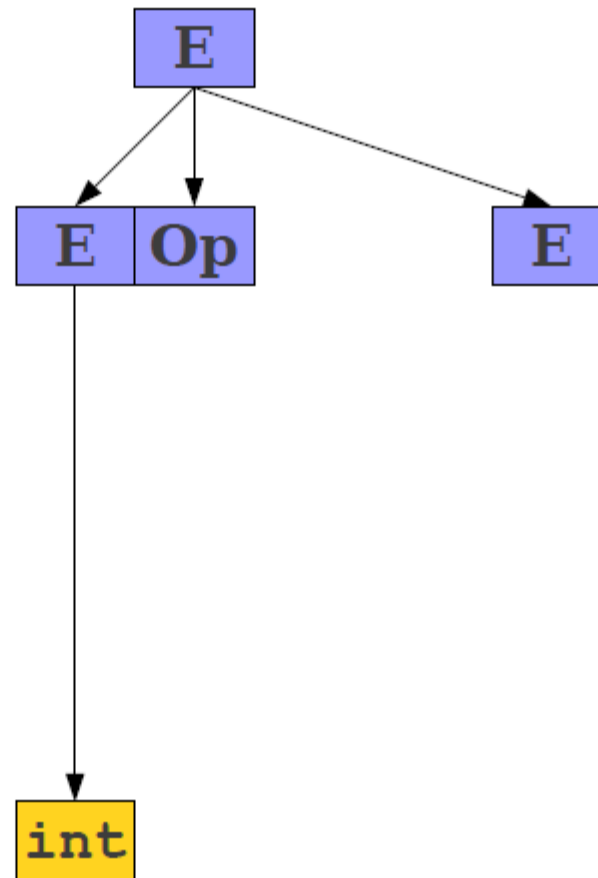
**E**  
⇒ **E Op E**  
⇒ **int Op E**





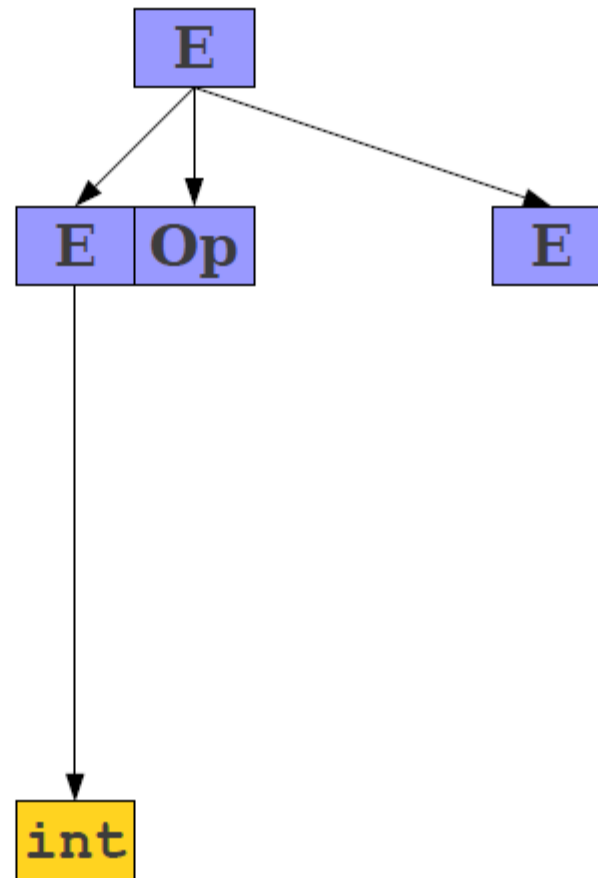
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**



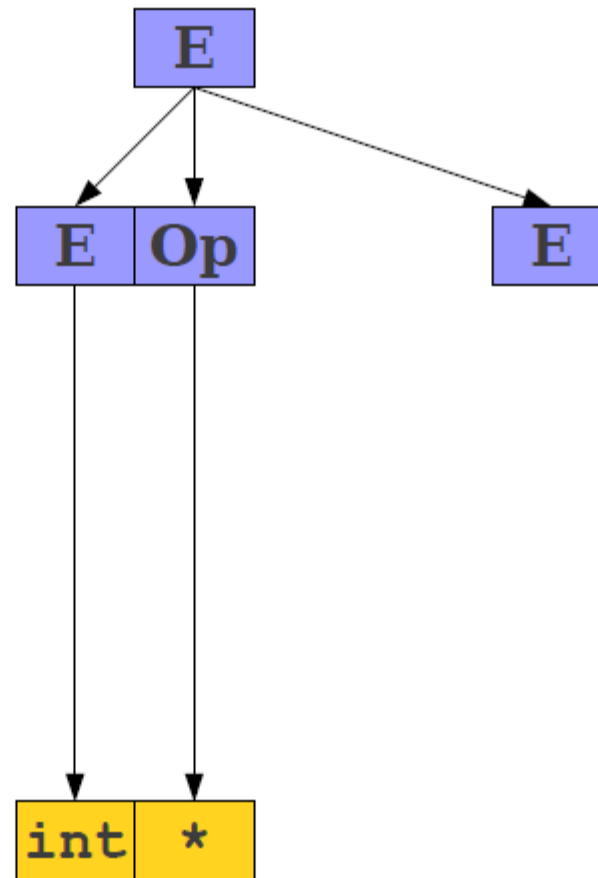
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**



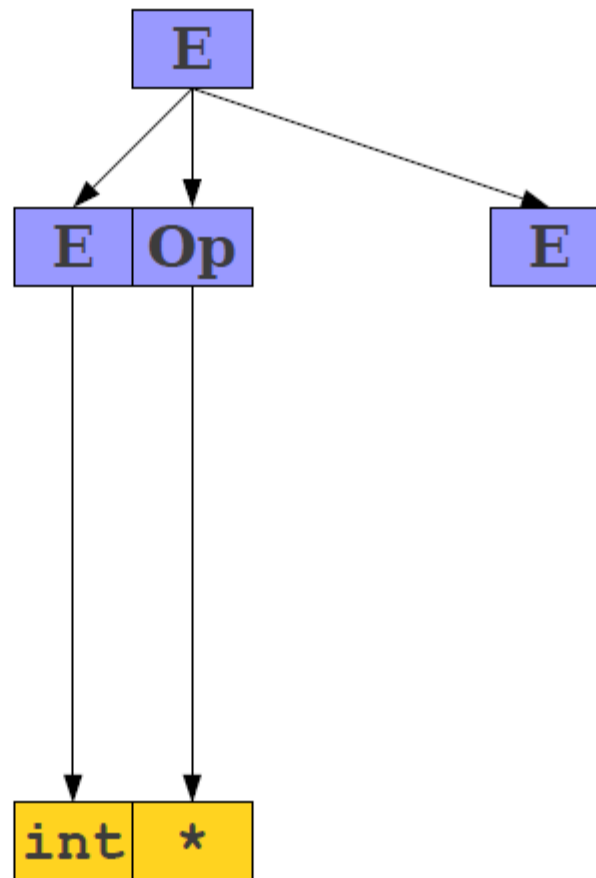
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**



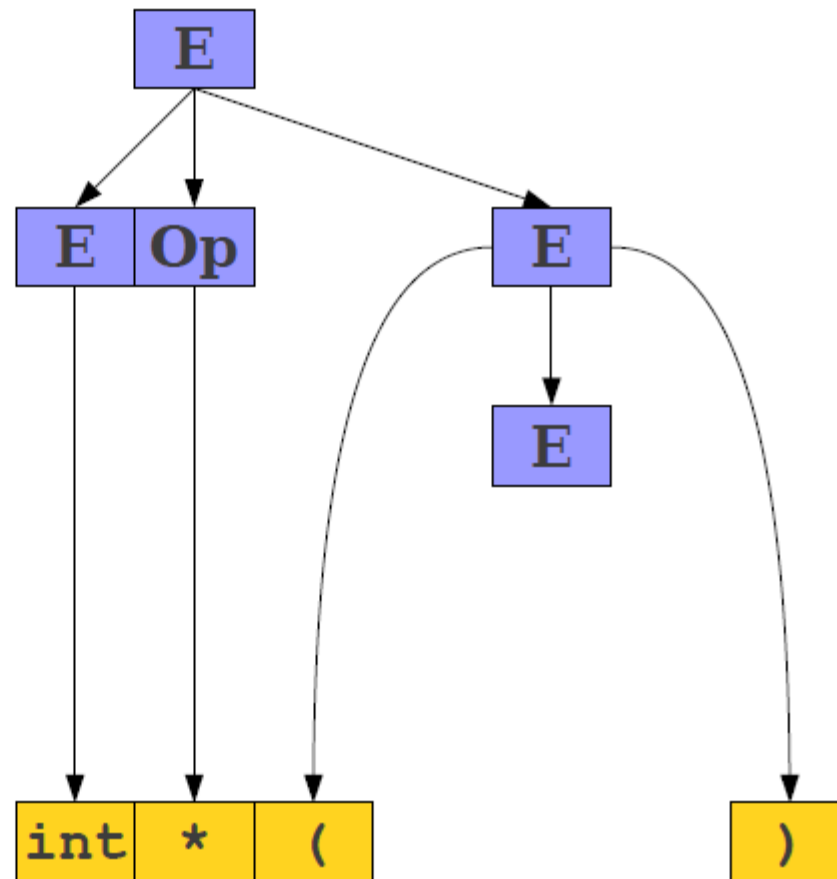
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**  
⇒ **int \* (E)**



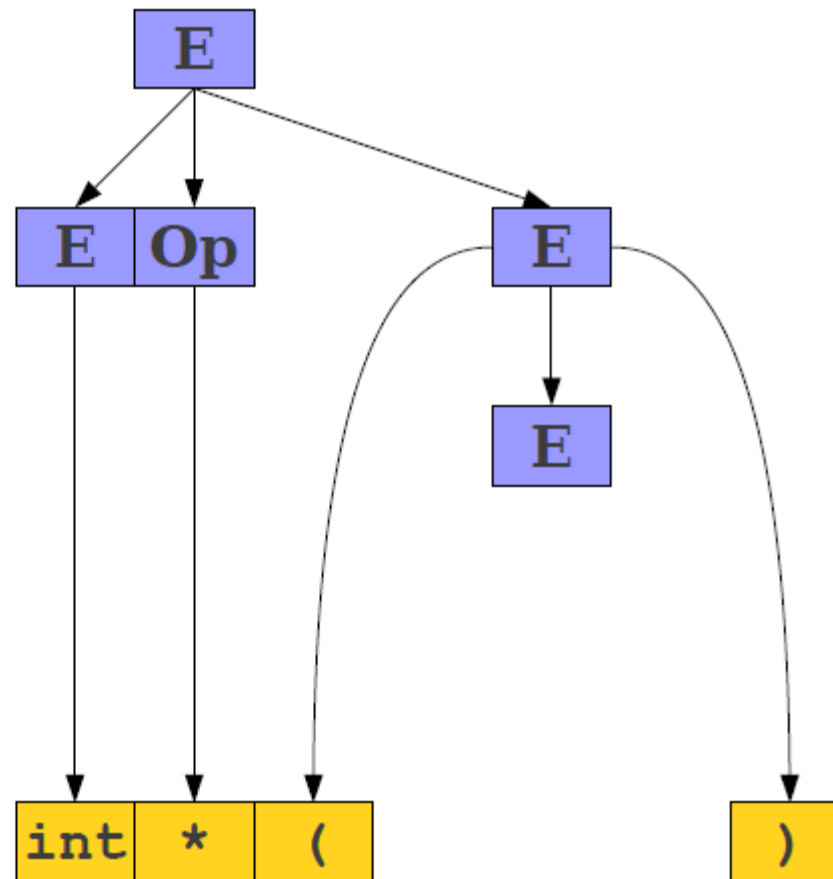
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**  
⇒ **int \* (E)**



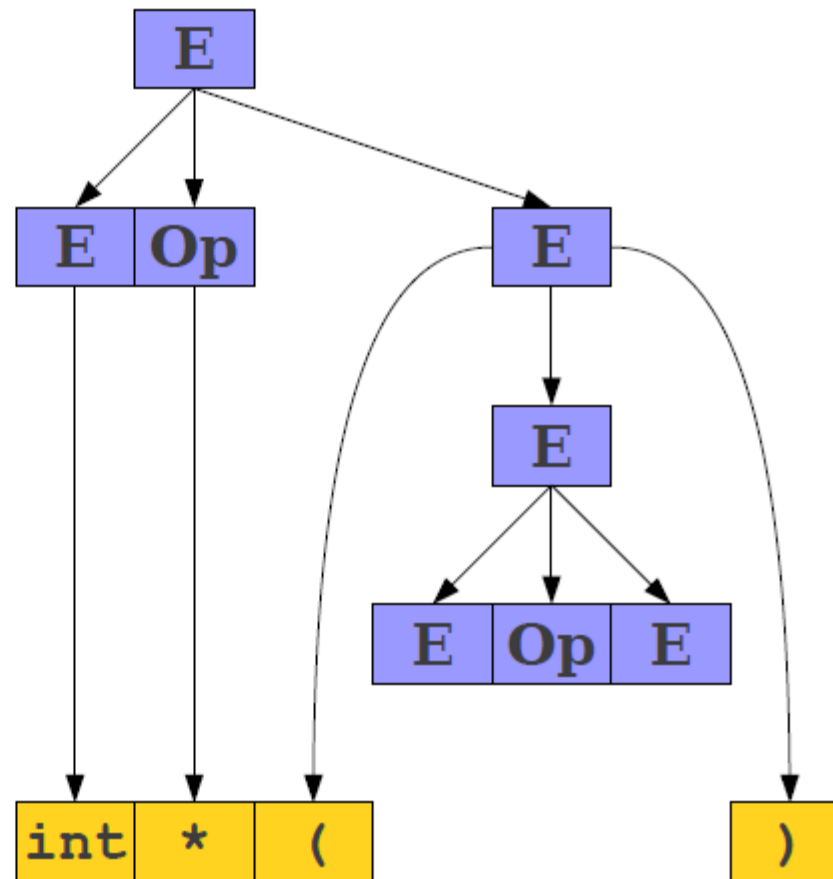
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**  
⇒ **int \* (E)**  
⇒ **int \* (E Op E)**



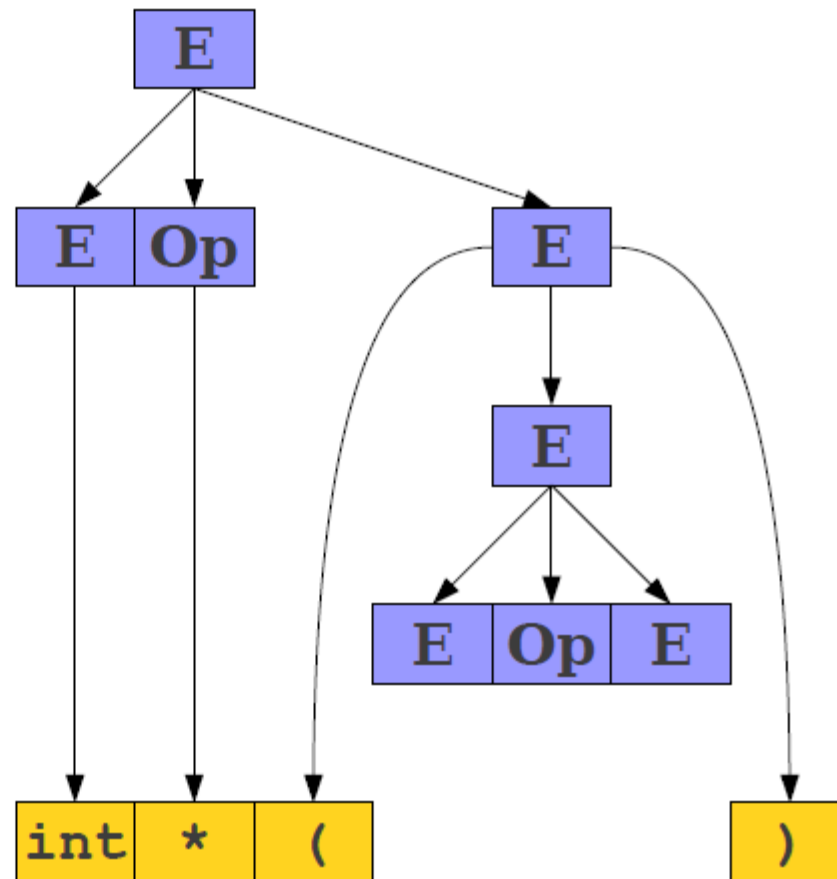
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**  
⇒ **int \* (E)**  
⇒ **int \* (E Op E)**



# Cây phân tích cú pháp

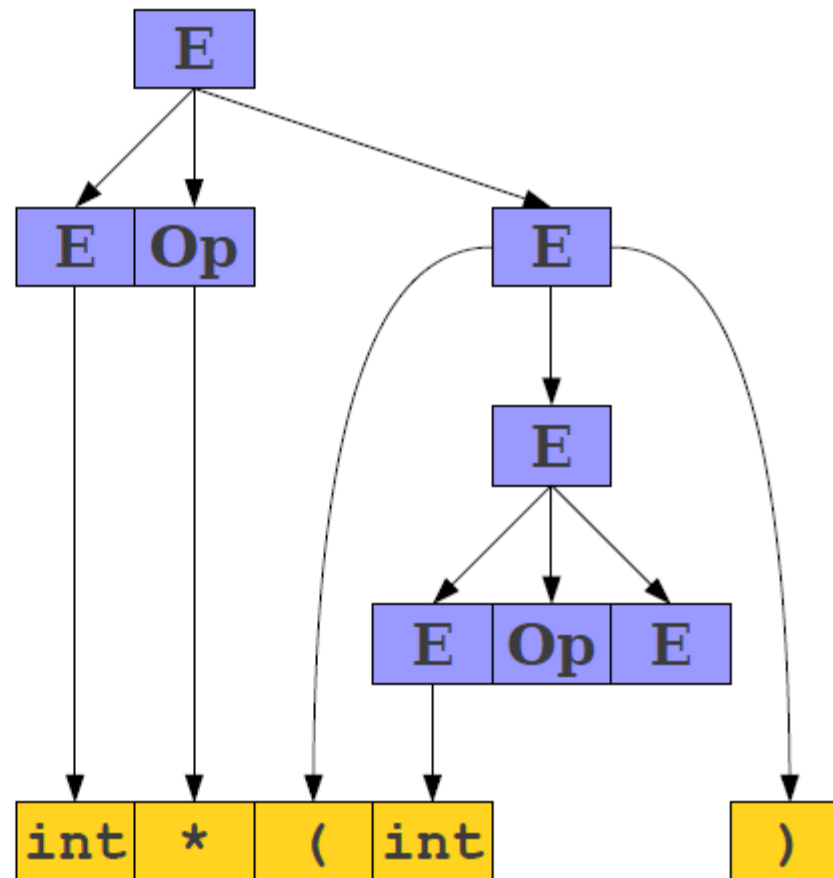
**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**  
⇒ **int \* (E)**  
⇒ **int \* (E Op E)**  
⇒ **int \* (int Op E)**





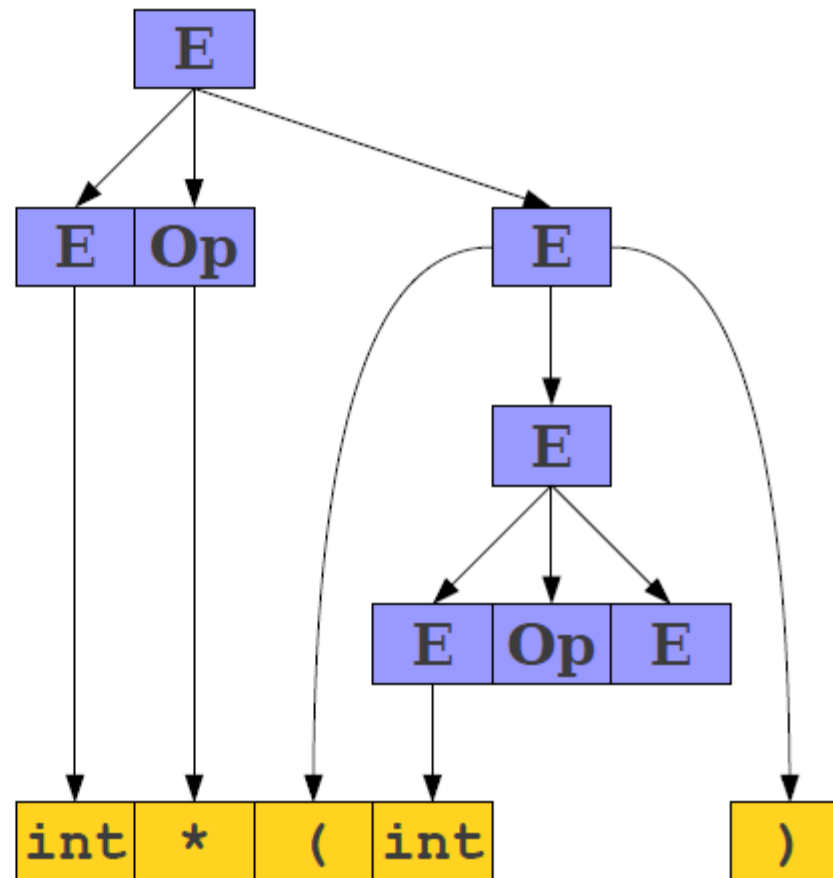
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**  
⇒ **int \* (E)**  
⇒ **int \* (E Op E)**  
⇒ **int \* (int Op E)**



# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ int **Op E**  
⇒ int \* **E**  
⇒ int \* (**E**)  
⇒ int \* (**E Op E**)  
⇒ int \* (int **Op E**)  
⇒ int \* (int + **E**)



# Cây phân tích cú pháp

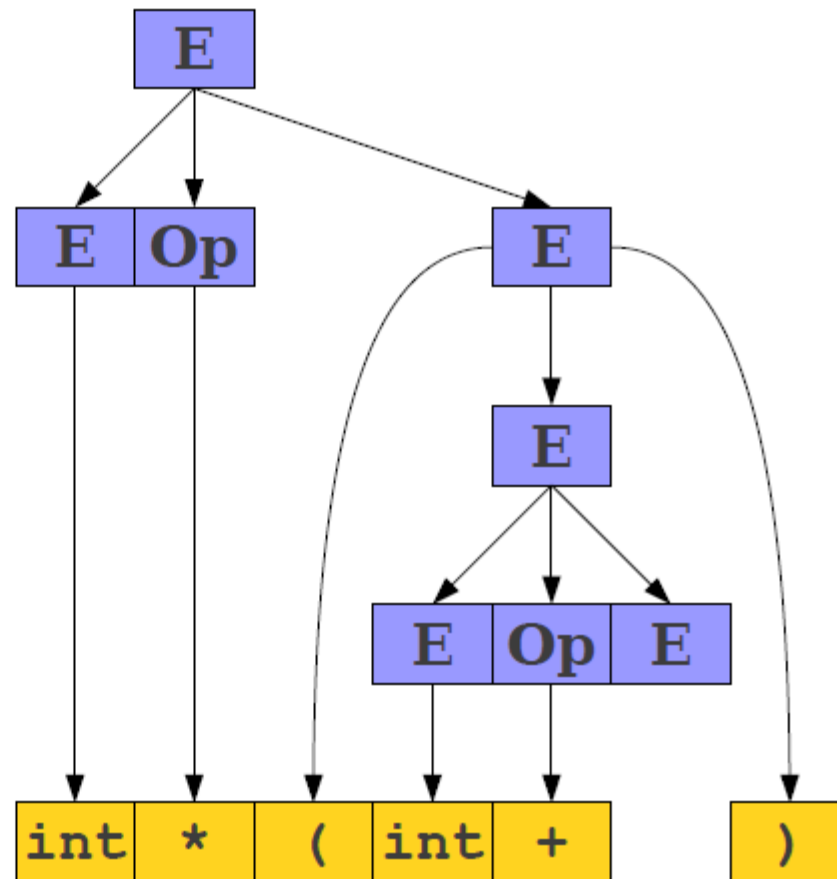
## E

$$\Rightarrow E \text{ Op } E$$

⇒ int Op E

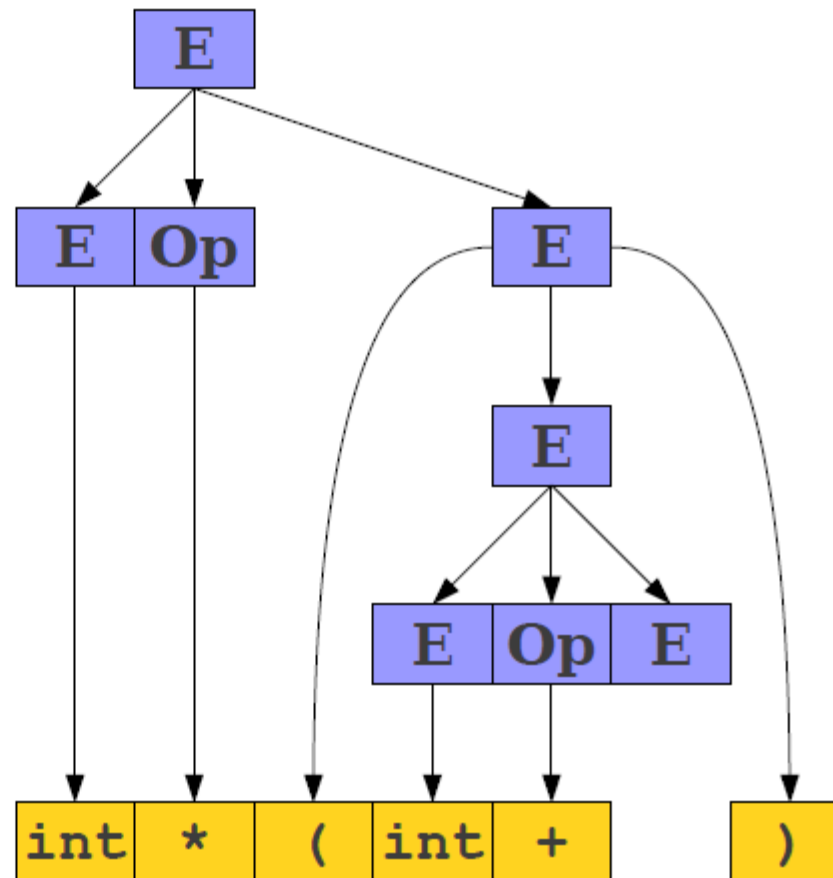
$$\Rightarrow \text{int} * \mathbf{E}$$

$\Rightarrow \text{int} * (\text{E})$

$$\Rightarrow \text{int} * (\text{E Op E})$$
$$\Rightarrow \text{int} * (\text{int Op E})$$
$$\Rightarrow \text{int} * (\text{int} + \mathbf{E})$$


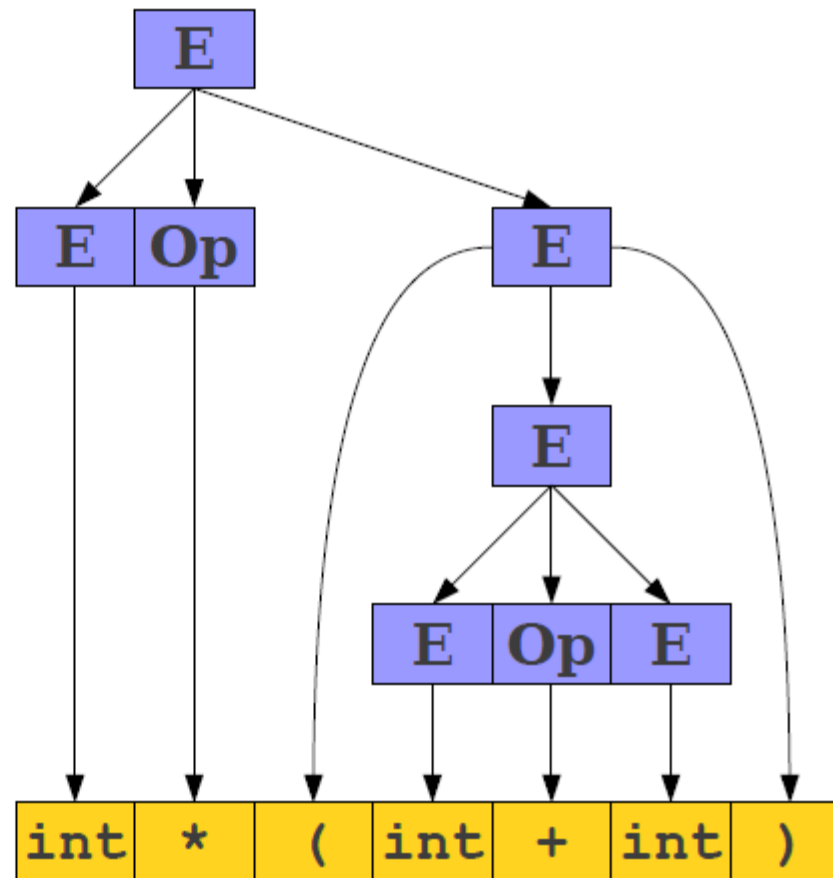
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ int **Op E**  
⇒ int \* **E**  
⇒ int \* (**E**)  
⇒ int \* (**E Op E**)  
⇒ int \* (int **Op E**)  
⇒ int \* (int + **E**)  
⇒ int \* (int + int)



# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **int Op E**  
⇒ **int \* E**  
⇒ **int \* (E)**  
⇒ **int \* (E Op E)**  
⇒ **int \* (int Op E)**  
⇒ **int \* (int + E)**  
⇒ **int \* (int + int)**



# Cây phân tích cú pháp

E

E

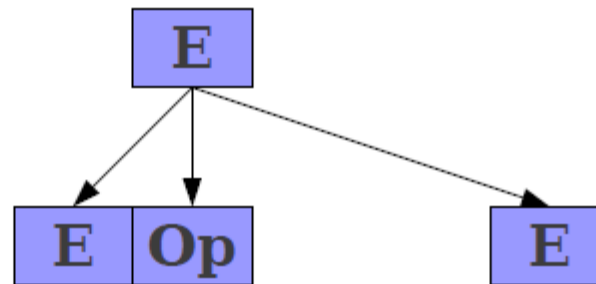
# Cây phân tích cú pháp

E

**E**  
 $\Rightarrow$  **E Op E**

# Cây phân tích cú pháp

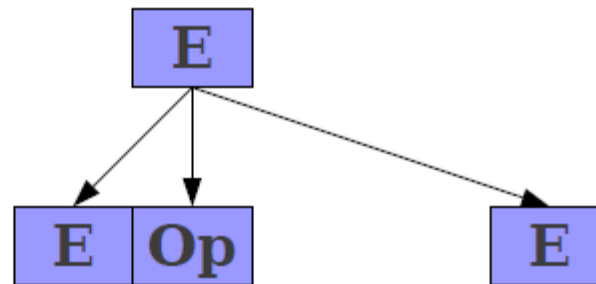
**E**  
 $\Rightarrow$  **E Op E**





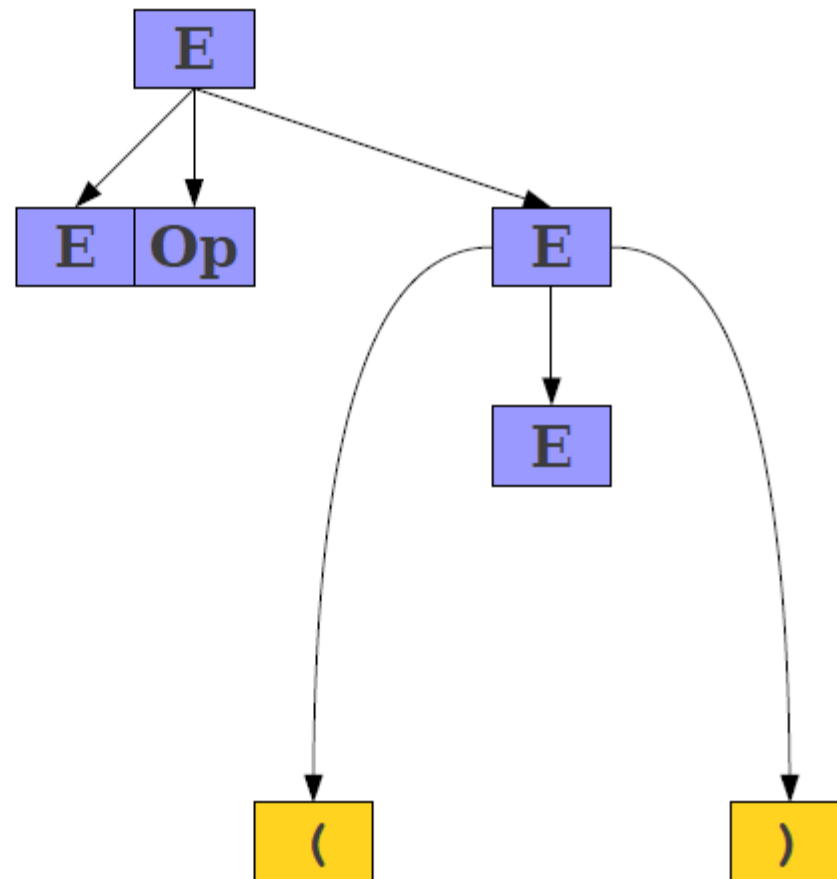
# Cây phân tích cú pháp

**E**  
 $\Rightarrow$  **E Op E**  
 $\Rightarrow$  **E Op (E)**



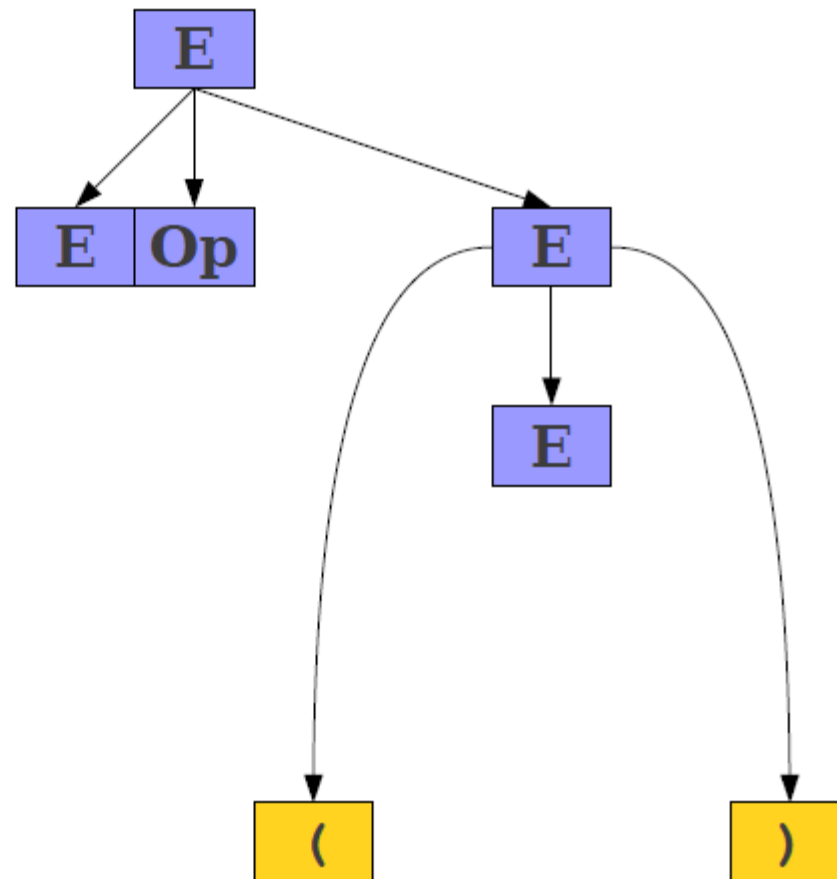
# Cây phân tích cú pháp

**E**  
 $\Rightarrow$  **E Op E**  
 $\Rightarrow$  **E Op (E)**



# Cây phân tích cú pháp

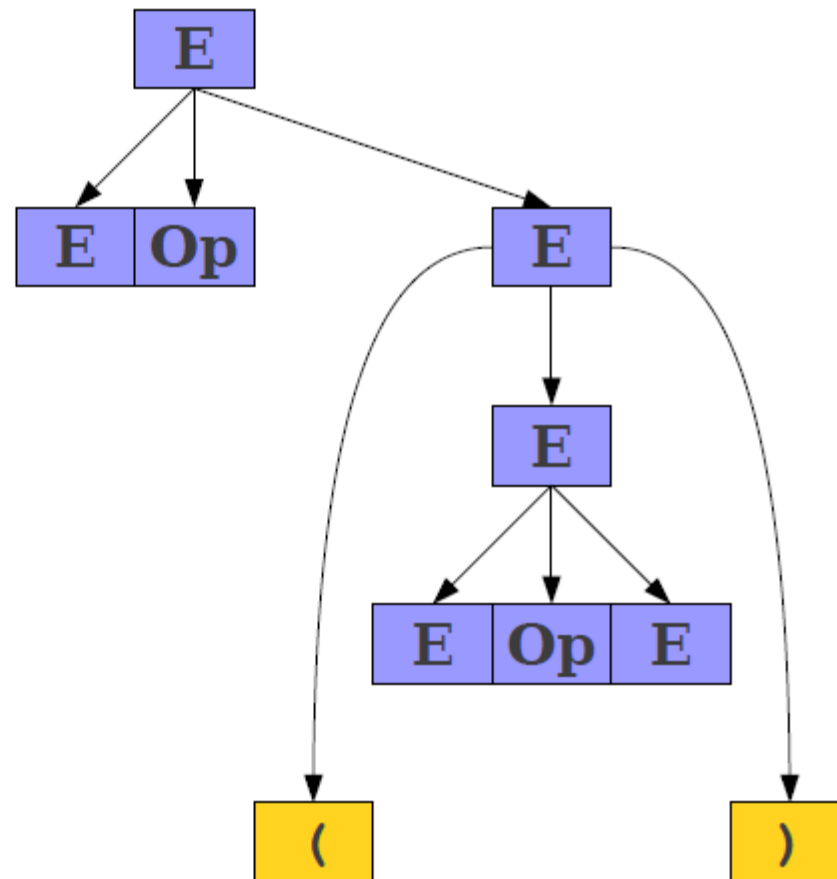
**E**  
 $\Rightarrow$  **E Op E**  
 $\Rightarrow$  **E Op (E)**  
 $\Rightarrow$  **E Op (E Op E)**





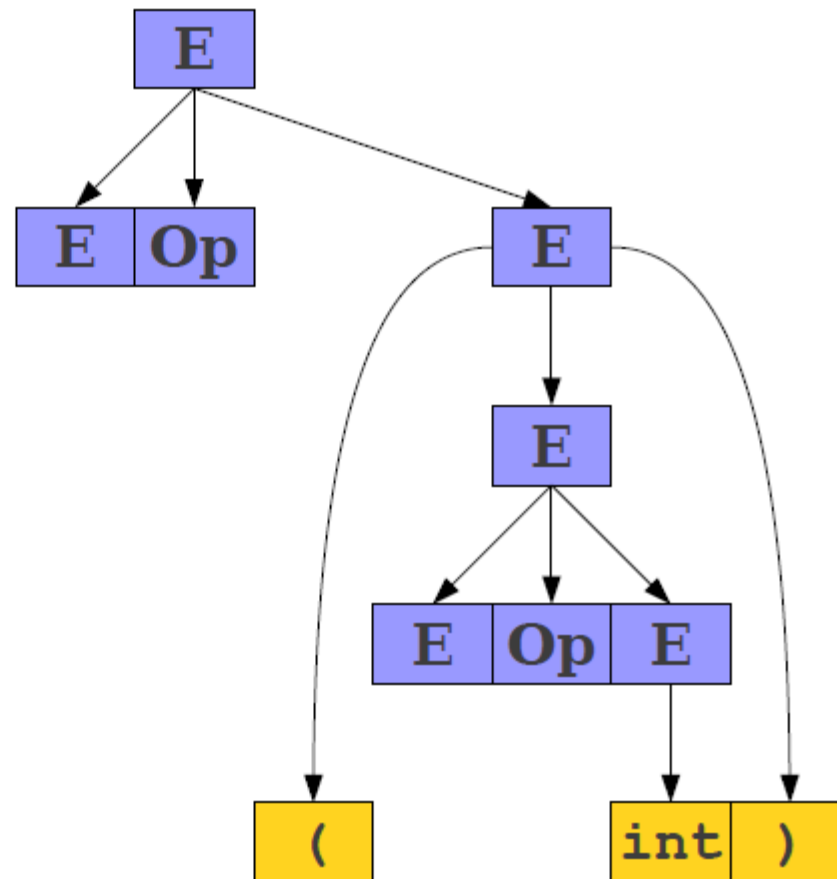
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**



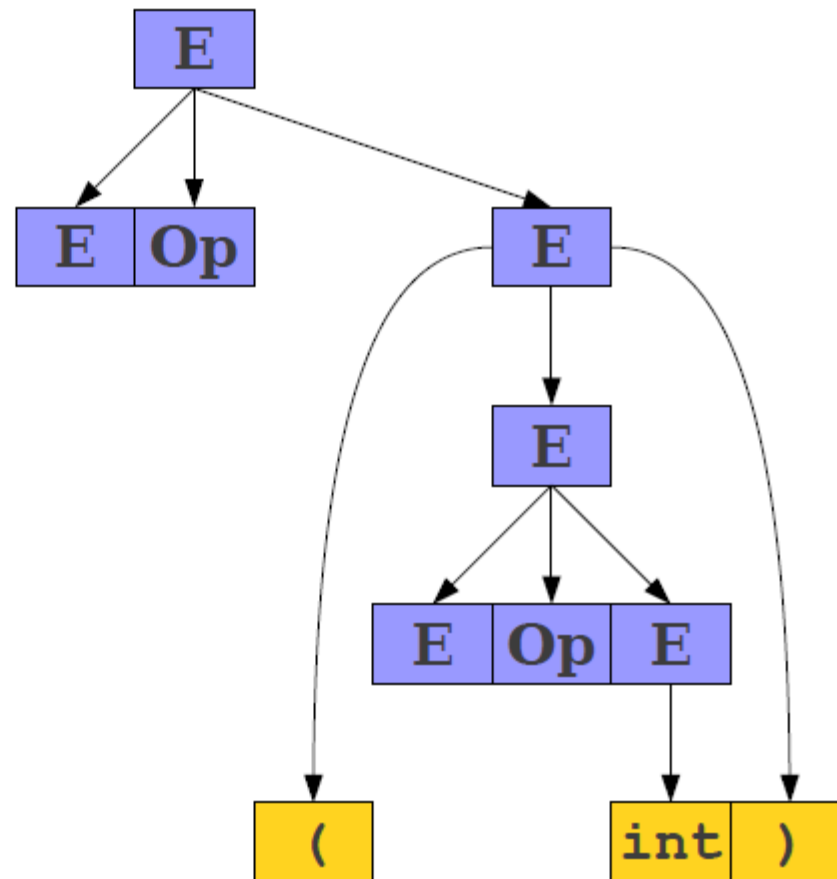
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**



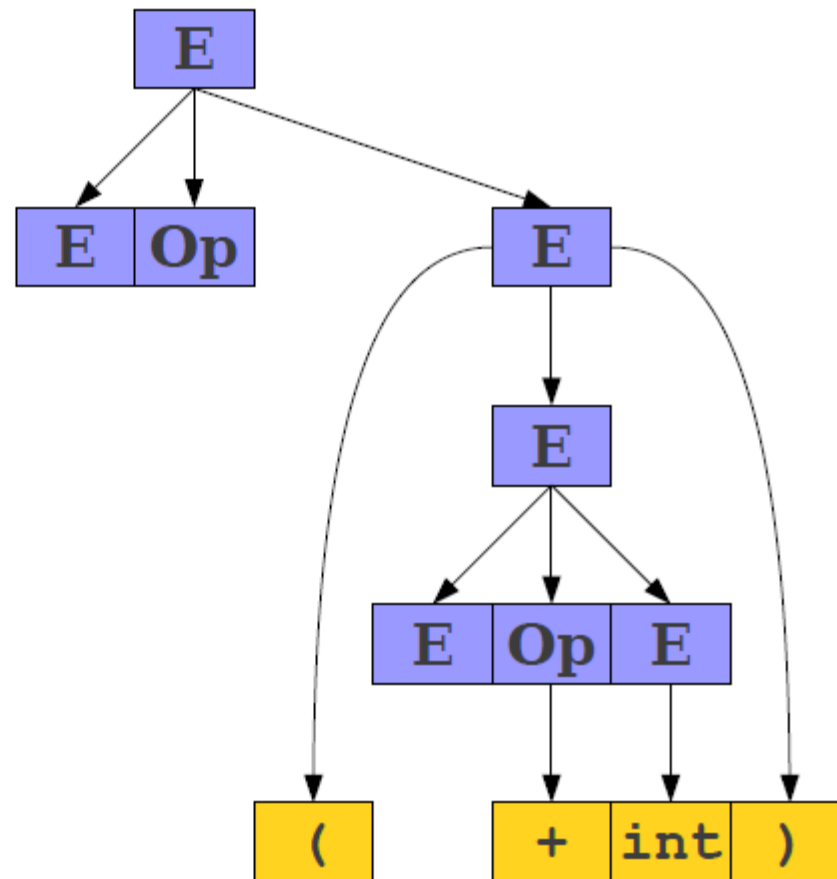
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**  
⇒ **E Op (E + int)**



# Cây phân tích cú pháp

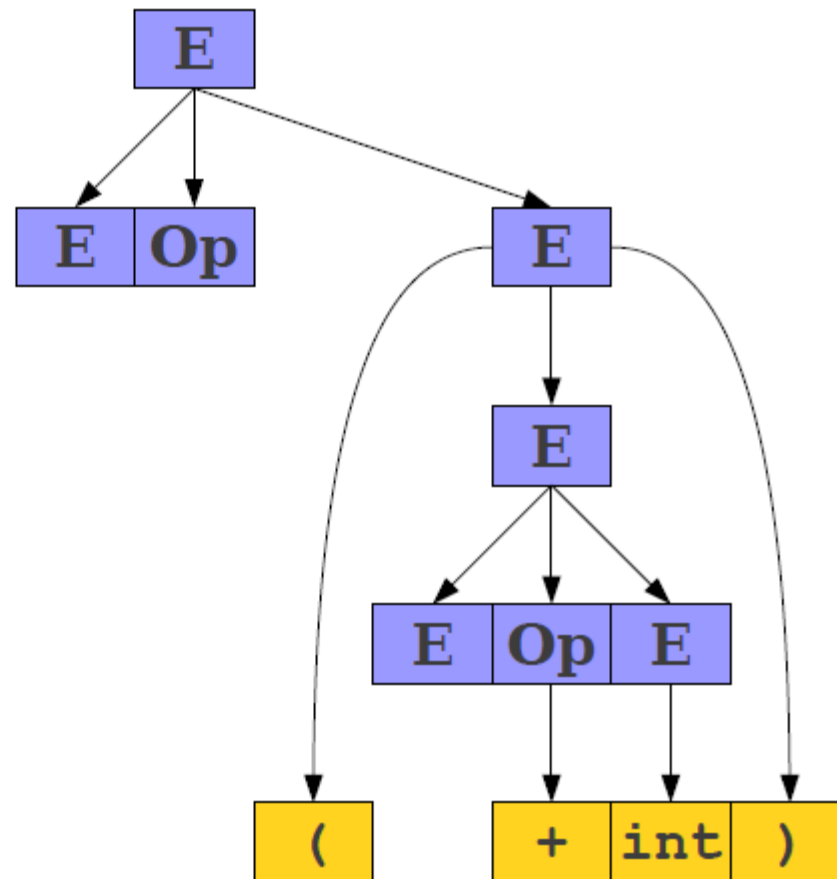
**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**  
⇒ **E Op (E + int)**





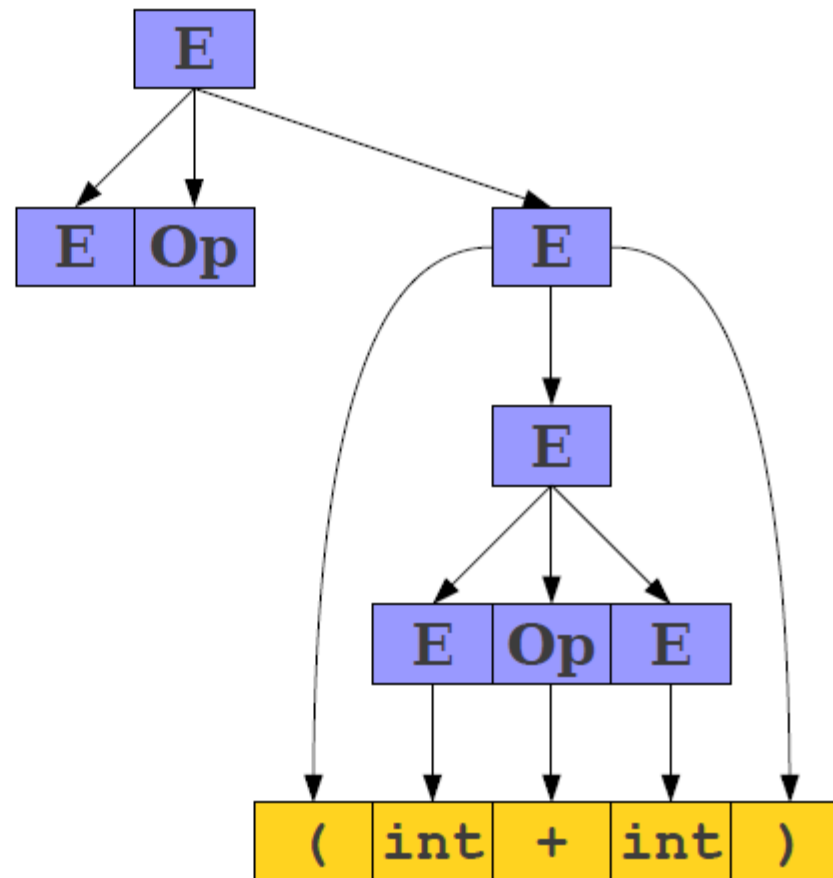
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**  
⇒ **E Op (E + int)**  
⇒ **E Op (int + int)**



# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**  
⇒ **E Op (E + int)**  
⇒ **E Op (int + int)**

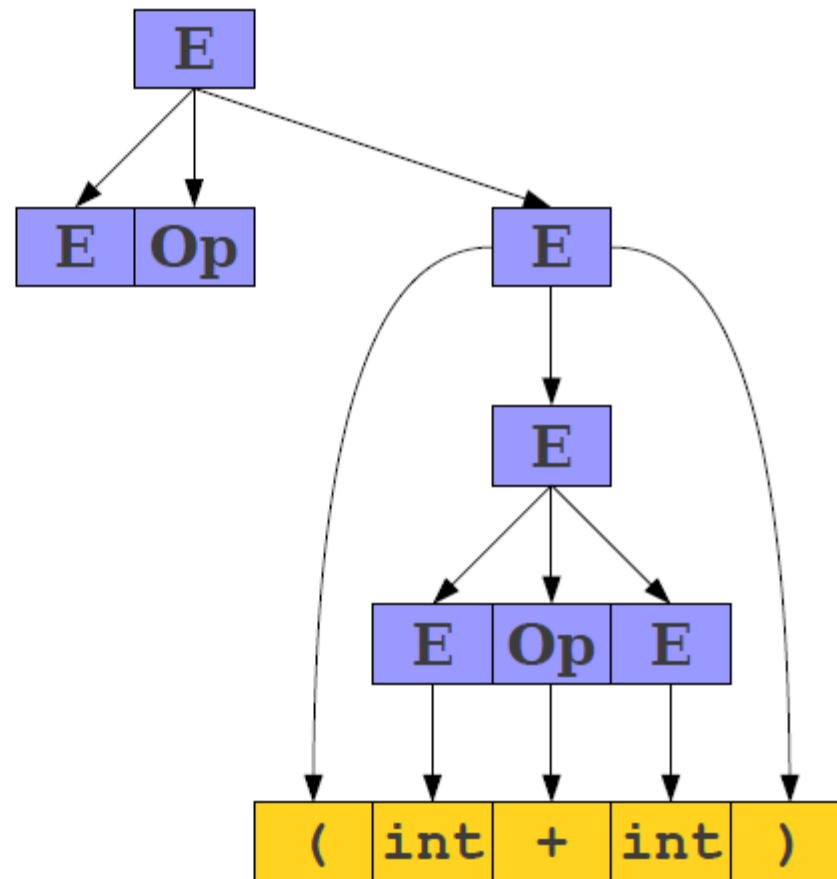


# Cây phân tích cú pháp

## E

$$\Rightarrow E \text{ Op } E$$
$$\Rightarrow E \text{ Op } (E)$$
$$\Rightarrow E \text{ Op } (E \text{ Op } E)$$
$$\Rightarrow E \text{ Op } (E \text{ Op int})$$
$$\Rightarrow E \text{ Op } (E + \text{int})$$

⇒ **E Op** (int + int)

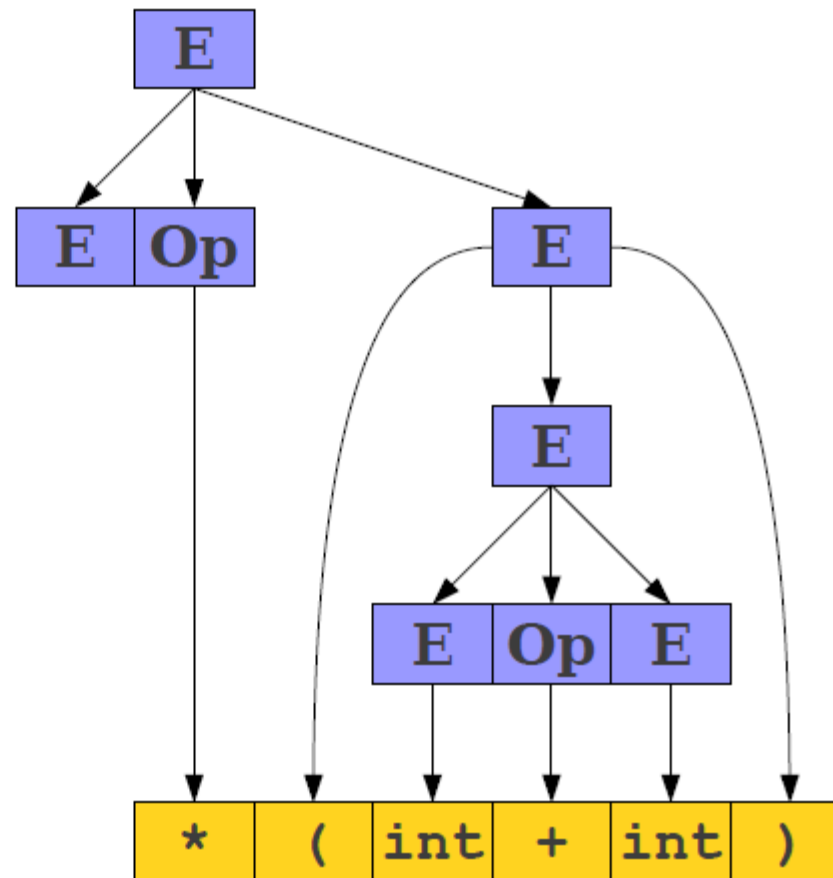
$$\Rightarrow E * (\text{int} + \text{int})$$


# Cây phân tích cú pháp

## E

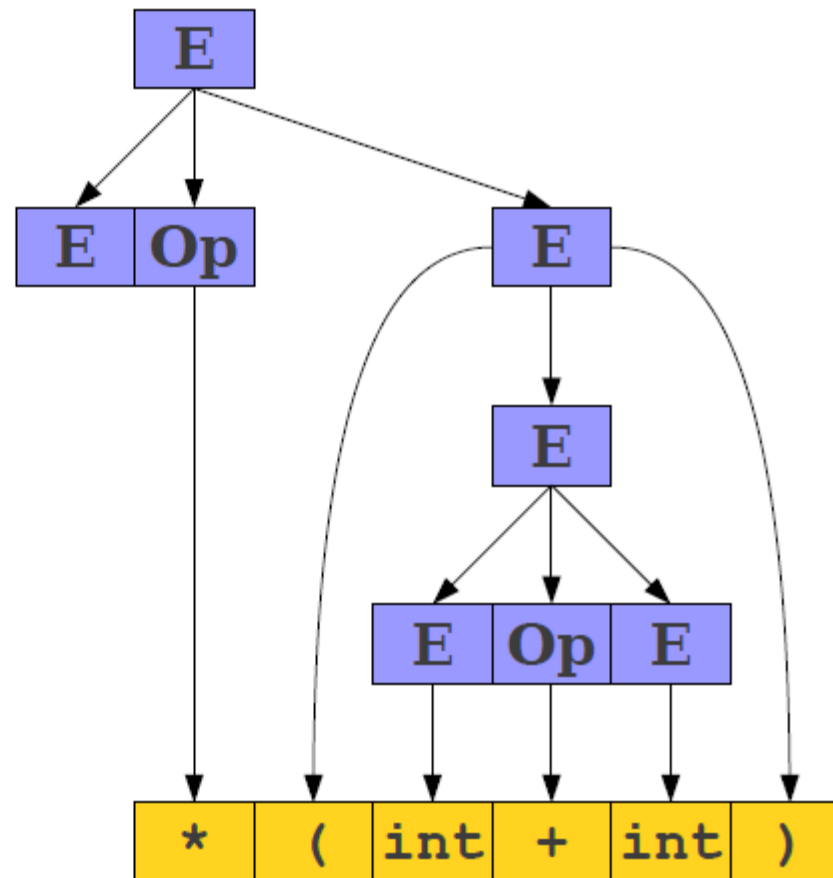
$$\Rightarrow E \text{ Op } E$$
$$\Rightarrow E \text{ Op } (E)$$
$$\Rightarrow E \text{ Op } (E \text{ Op } E)$$
$$\Rightarrow E \text{ Op } (E \text{ Op int})$$
$$\Rightarrow E \text{ Op } (E + \text{int})$$

⇒ **E Op** (int + int)

$$\Rightarrow E * (\text{int} + \text{int})$$


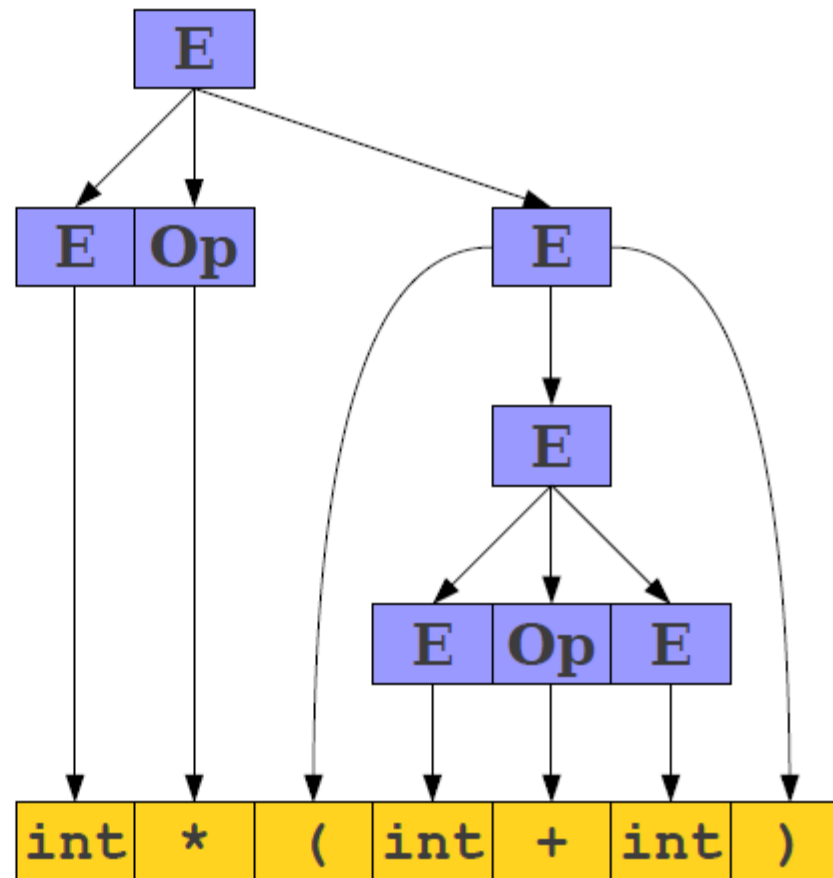
# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**  
⇒ **E Op (E + int)**  
⇒ **E Op (int + int)**  
⇒ **E \* (int + int)**  
⇒ **int \* (int + int)**

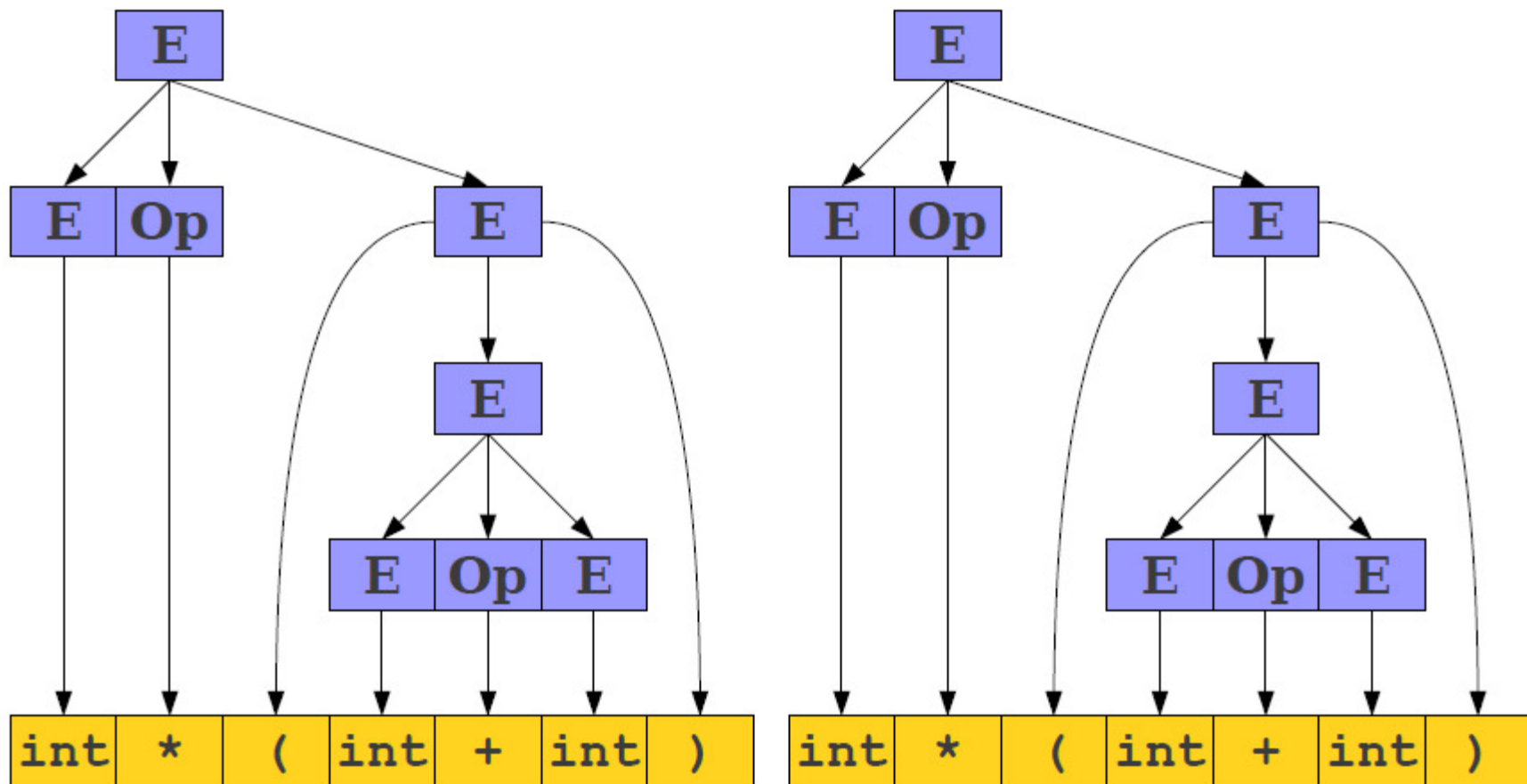


# Cây phân tích cú pháp

**E**  
⇒ **E Op E**  
⇒ **E Op (E)**  
⇒ **E Op (E Op E)**  
⇒ **E Op (E Op int)**  
⇒ **E Op (E + int)**  
⇒ **E Op (int + int)**  
⇒ **E \* (int + int)**  
⇒ **int \* (int + int)**



# So sánh 2 cây phân tích cú pháp



# Cây phân tích cú pháp

- **Cây phân tích cú pháp (parse tree)** là một cây mã hóa các bước trong một dẫn xuất.
- Các nút trong của cây biểu diễn các ký hiệu chưa kết thúc được sử dụng trong dẫn xuất.
- Duyệt trung tự các nút lá sẽ thu được chuỗi kết quả.
- Chỉ mã hóa luật sinh nào được sử dụng nhưng không mã hóa thứ tự áp dụng các luật sinh.

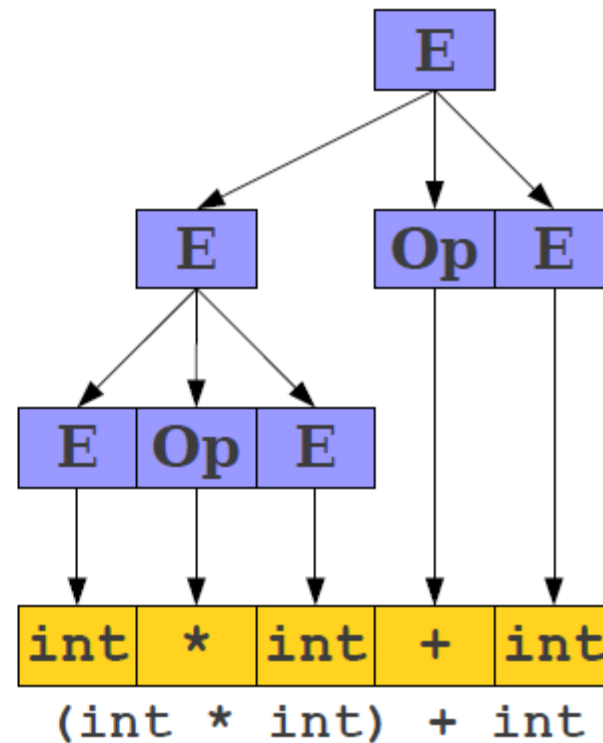
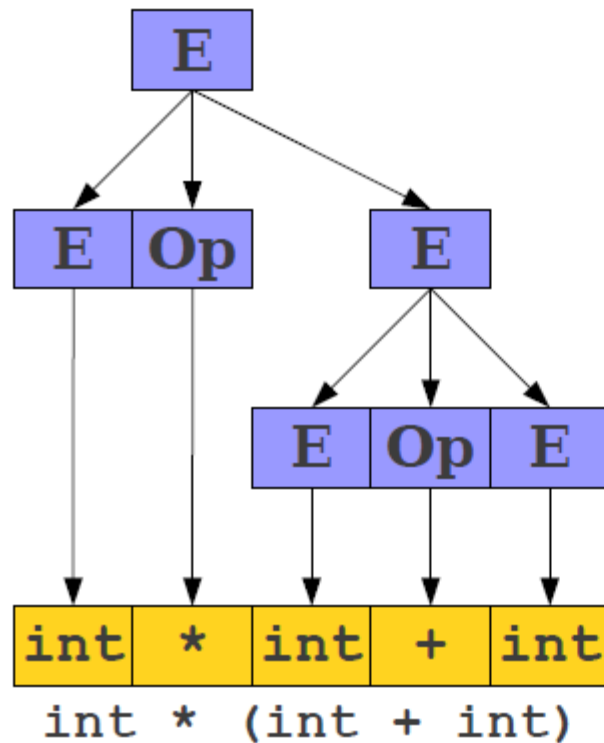


# Mục tiêu của phân tích cú pháp

- Dựng lại **cấu trúc mà chuỗi các tokens mô tả**.
- Nếu ngôn ngữ được mô tả bằng một văn phạm phi ngữ cảnh, mục tiêu của phân tích cú pháp là:
- Xây dựng cây phân tích cú pháp cho chuỗi đầu vào.
- Thông thường, ta đơn giản hóa cây phân tích cú pháp.

Khó khăn trong phân tích cú pháp

# Một vấn đề nghiêm trọng



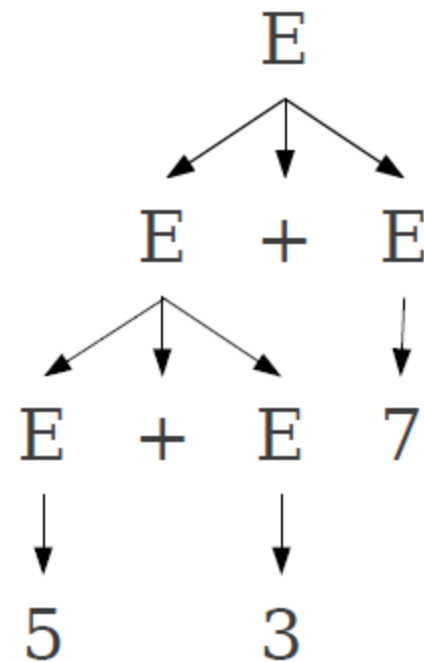
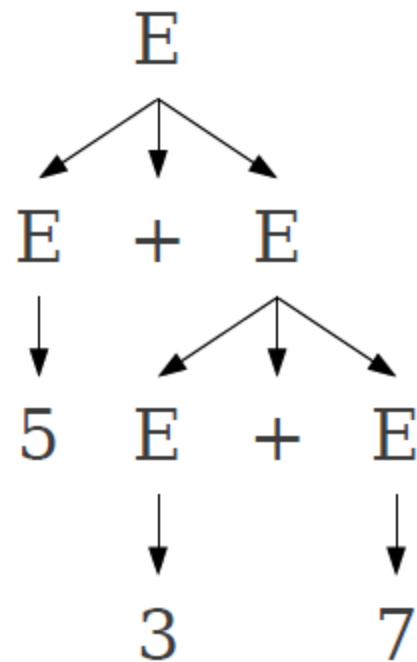
# Mơ hồ/nhập nhằng

- Một văn phạm phi ngữ cảnh được nói là **mơ hồ/nhập nhằng** nếu có ít nhất một chuỗi đầu vào mà có 2 hoặc nhiều hơn cây phân tích cú pháp khác nhau.
- Chú ý rằng mơ hồ này là do **văn phạm** chứ không phải do **ngôn ngữ**.
- Không có giải thuật nào có thể biến đổi một văn phạm mơ hồ bất kỳ thành văn phạm không mơ hồ.
  - Một số ngôn ngữ bản thân nó đã mơ hồ, nghĩa là không tồn tại văn phạm không mơ hồ để mô tả nó.
- Không có giải thuật nào có thể phát hiện một văn phạm bất kỳ có mơ hồ hay không.

# Mơ hồ có là vấn đề không ?

- Tùy vào **ngữ nghĩa**

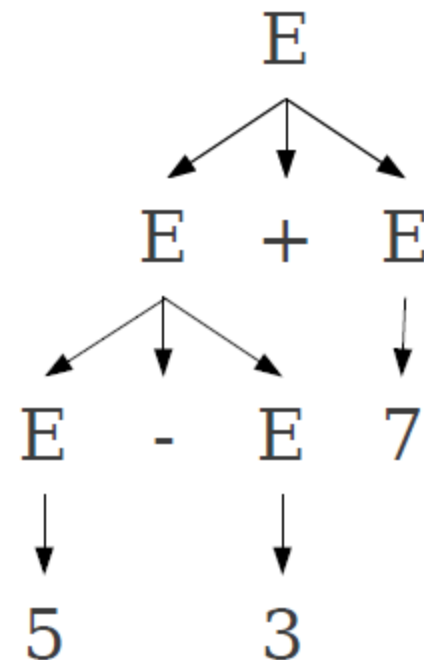
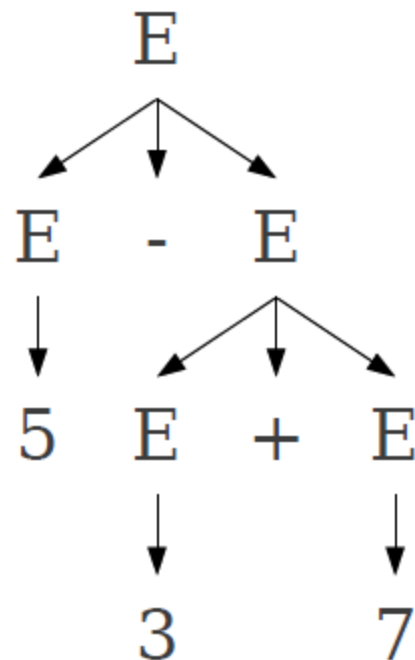
**E**  $\rightarrow$  **int** | **E + E**



# Mơ hồ có là vấn đề không ?

- Tùy vào **ngữ nghĩa**

$E \rightarrow \text{int} \mid E + E \mid E - E$



# Khử mơ hồ

- Nếu văn phạm có thể khử mơ hồ được, ta có thể khử nó bằng phương pháp **phân tầng (layering)**.
- Phân tầng: biến đổi văn phạm sao cho
  - chỉ có một cách duy nhất xây dựng mỗi phần của chuỗi đầu vào;
  - chỉ có một cách duy nhất ghép các phần của chuỗi đầu vào

## Ví dụ: chuỗi dấu ngoặc cân bằng

- Xét ngôn ngữ bao gồm tất cả các chuỗi dấu ngoặc cân bằng.
- Ví dụ:
  - $\epsilon$  chuỗi rỗng
  - $()$
  - $((()))$
  - $((()))(())()$
- Một trong các văn phạm phi ngữ cảnh cho ngôn ngữ này là:



## Ví dụ: chuỗi dấu ngoặc cân bằng

- Xét ngôn ngữ bao gồm tất cả các chuỗi dấu ngoặc cân bằng.
- Ví dụ:
  - $\epsilon$  chuỗi rỗng
  - $()$
  - $((()))$
  - $((()))(())()$
- Một trong các văn phạm phi ngữ cảnh cho ngôn ngữ này là:

$$P \rightarrow \epsilon \mid PP \mid (P)$$

## Ví dụ: chuỗi dấu ngoặc cân bằng

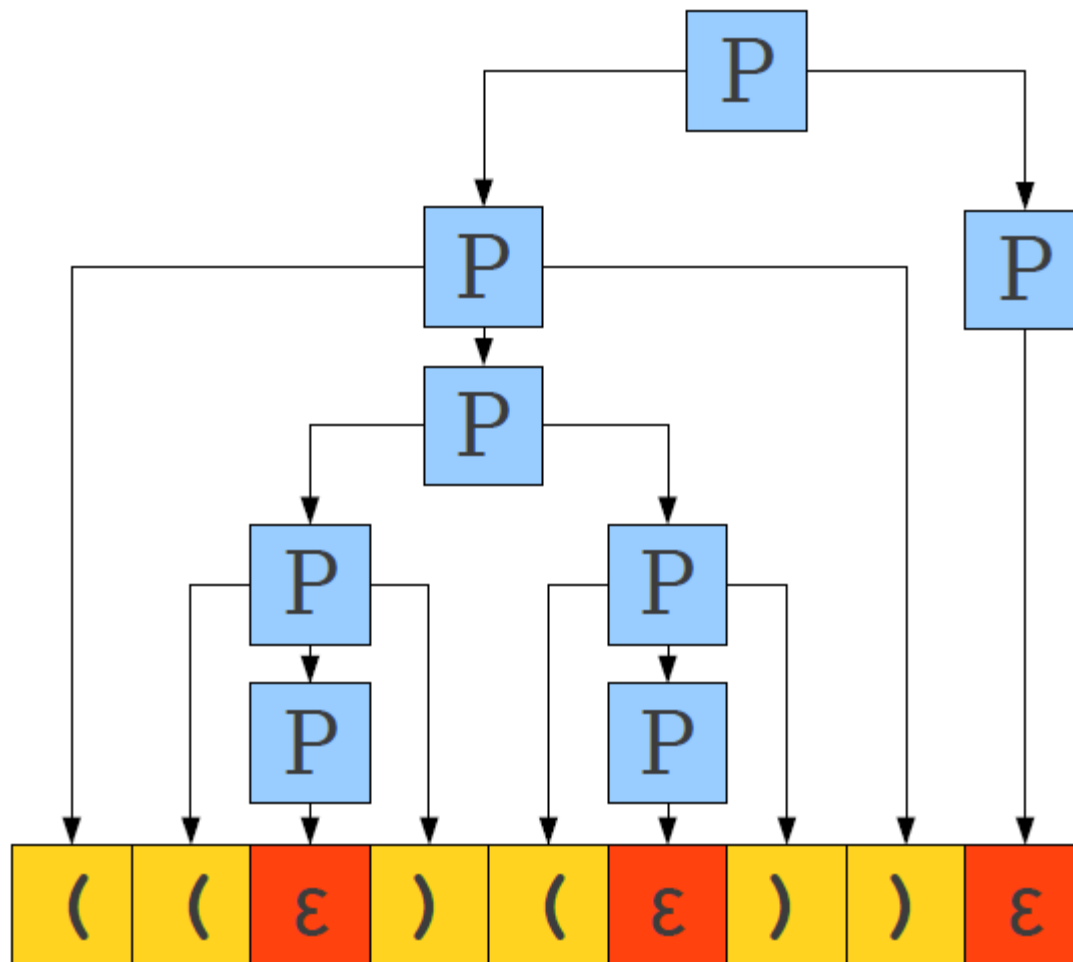
- Cho văn phạm:

$$P \rightarrow \epsilon \mid PP \mid (P)$$

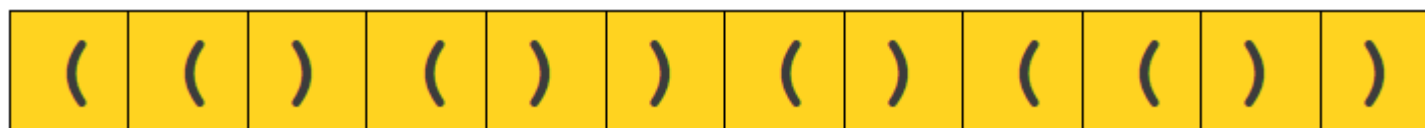
- Tìm một cây phân tích cú pháp sinh ra chuỗi  
(())



Ví dụ: chuỗi dấu ngoặc cân bằng



Khử nhập nhằng cho văn  
phạm này như thế nào ?



## Chia chuỗi nhập thành từng phần có cấu trúc giống nhau

Trong trường hợp này, ta thấy chuỗi đầu vào được xây dựng từ việc ghép **3 chuỗi dấu ngoặc cân bằng**

( ( ) ( ) ) ( ) ( ( ) )

## Quan sát từng thành phần để tìm cấu trúc của nó

Ta thấy phần thứ nhất được xây dựng bằng cách bao một chuỗi dấu ngoặc cân bằng với cặp dấu ngoặc





## Quan sát từng thành phần để tìm cấu trúc của nó

Ta thấy phần thứ nhất được xây dựng bằng cách bao một chuỗi dấu ngoặc cân bằng với cặp dấu ngoặc

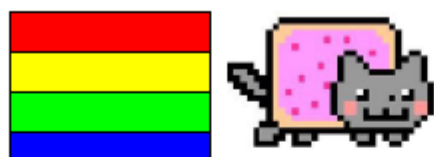


# Chuỗi dấu ngoặc cân bằng

- Một chuỗi dấu ngoặc cân bằng là dãy các chuỗi dấu ngoặc cân bằng
- Để tránh mơ hồ, ta có thể xây dựng chuỗi này theo 2 bước:
  - Xét xem có bao nhiêu chuỗi con có cấu trúc khác nhau có thể ghép lại với nhau
  - Xây dựng mỗi chuỗi con một cách độc lập

**Để xây dựng văn phạm cho phép ghép các thành phần lại với nhau ta dựa trên nguyên lý sau:**

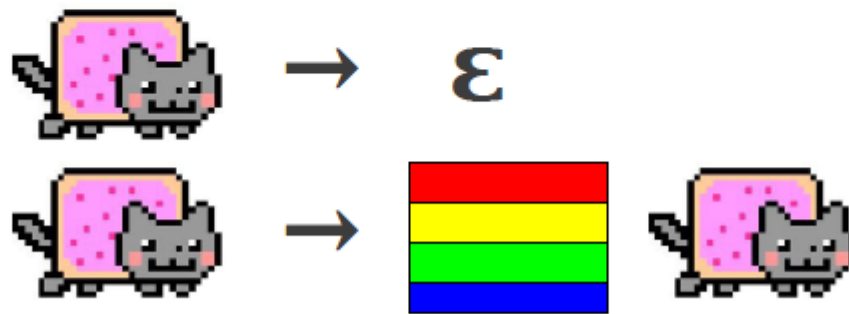














# Xây dựng chuỗi dấu ngoặc

- Trải rộng chuỗi các dấu ngoặc cân bằng. Chỉ có một cách duy nhất thực hiện điều này.
- Triển khai các chuỗi con bằng cách bao dấu ngoặc xung quanh nó, và cứ thế ...

$$S \rightarrow P S \mid \epsilon$$

$$P \rightarrow ( S )$$

# Xây dựng chuỗi dấu ngoặc

**S**  $\rightarrow$  **P** **S** |  $\epsilon$

**P**  $\rightarrow$  ( **S** )

**S**  
 $\Rightarrow$  **PS**  
 $\Rightarrow$  **PPS**  
 $\Rightarrow$  **PP**  
 $\Rightarrow$  (**S**) **P**  
 $\Rightarrow$  (**S**) (**S**)  
 $\Rightarrow$  (**PS**) (**S**)  
 $\Rightarrow$  (**P**) (**S**)  
 $\Rightarrow$  ((**S**)) (**S**)  
 $\Rightarrow$  (( )) (**S**)  
 $\Rightarrow$  (( )) ( )

# Văn phạm phi ngữ cảnh cho các biểu thức chính quy

- Một biểu thức chính quy có thể là:
  - Một ký tự bất kỳ
  - $\epsilon$
  - Kết quả việc ghép hai biểu thức chính quy
  - Hợp của hai biểu thức chính quy
  - Bao đóng Kleene của một biểu thức chính quy (phép toán  $*$ )
  - Một biểu thức chính quy được bao bằng cặp dấu ngoặc.

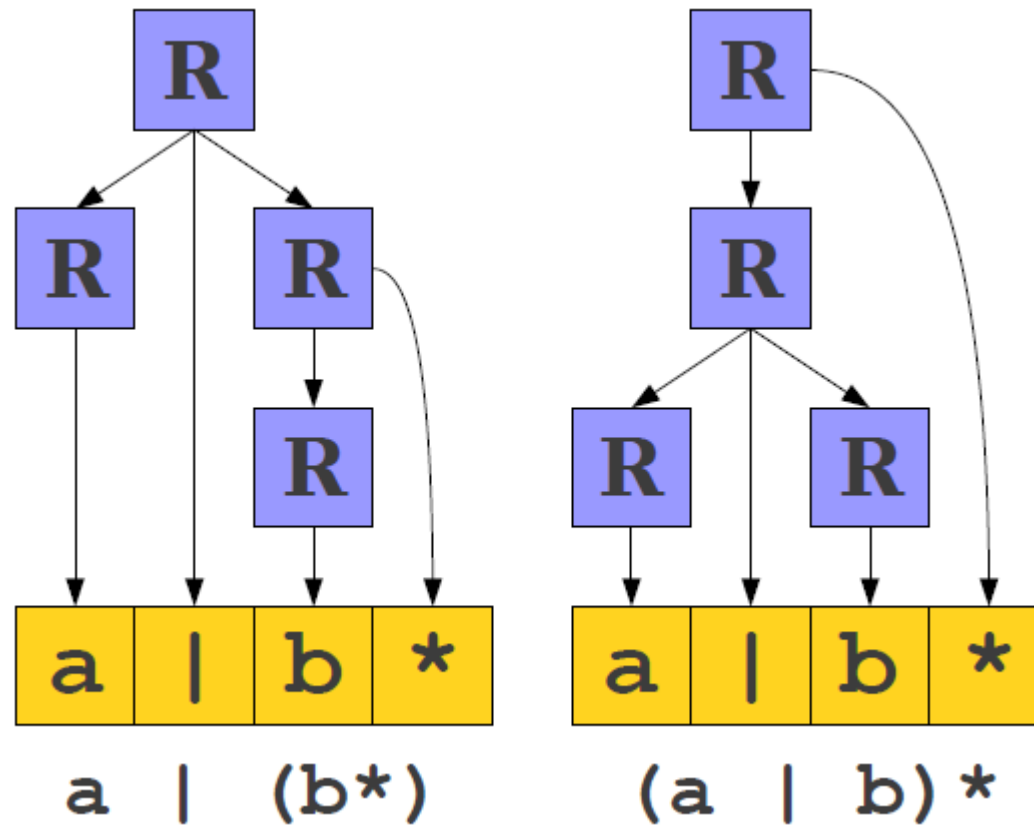
# Văn phạm phi ngữ cảnh cho các biểu thức chính quy

- Ta có thể xây dựng văn phạm cho các biểu thức chính quy như sau:
  - $R \rightarrow a \mid b \mid c \mid \dots$
  - $R \rightarrow \epsilon$
  - $R \rightarrow RR$
  - $R \rightarrow R \mid R$
  - $R \rightarrow R^*$
  - $R \rightarrow (R)$

# Văn phạm phi ngữ cảnh cho các biểu thức chính quy

- Ta có thể xây dựng văn phạm cho các biểu thức chính quy như sau:
  - $R \rightarrow a \mid b \mid c \mid \dots$
  - $R \rightarrow \epsilon$
  - $R \rightarrow RR$
  - $R \rightarrow R \mid R$
  - $R \rightarrow R^*$
  - $R \rightarrow (R)$
- Văn phạm này mơ hồ
  - Hãy tìm một ví dụ !

Văn phạm này mơ hồ



# Khử mơ hồ

$R \rightarrow a \mid b \mid c \mid \dots$

$R \rightarrow \epsilon$

$R \rightarrow RR$

$R \rightarrow R \mid R$

$R \rightarrow R^*$

$R \rightarrow (R)$

|   |   |  |   |   |
|---|---|--|---|---|
| a | a |  | b | * |
|---|---|--|---|---|

# Khử mơ hồ

$R \rightarrow a \mid b \mid c \mid \dots$

$R \rightarrow \epsilon$

$R \rightarrow RR$

$R \rightarrow R \mid R$

$R \rightarrow R^*$

$R \rightarrow (R)$

|   |   |  |   |   |
|---|---|--|---|---|
| a | a |  | b | * |
|---|---|--|---|---|



# Khử mơ hồ

$R \rightarrow a \mid b \mid c \mid \dots$

$R \rightarrow \epsilon$

$R \rightarrow RR$

$R \rightarrow R \mid R$

$R \rightarrow R^*$

$R \rightarrow (R)$

|   |   |  |   |   |
|---|---|--|---|---|
| a | a |  | b | * |
|---|---|--|---|---|

# Khử mơ hồ

$R \rightarrow a \mid b \mid c \mid \dots$

$R \rightarrow \epsilon$

$R \rightarrow RR$

$R \rightarrow R \mid R$

$R \rightarrow R^*$

$R \rightarrow (R)$



# Khử mơ hồ

$R \rightarrow a \mid b \mid c \mid \dots$

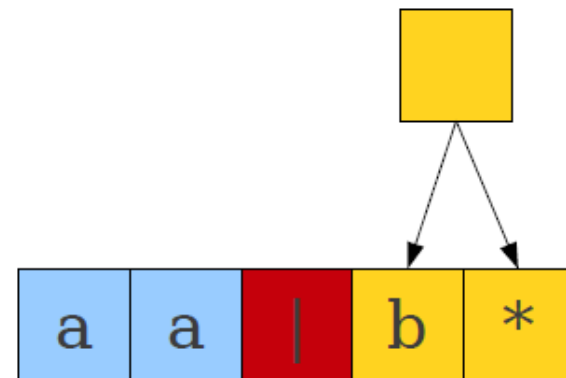
$R \rightarrow \epsilon$

$R \rightarrow RR$

$R \rightarrow R \mid R$

$R \rightarrow R^*$

$R \rightarrow (R)$



# Khử mơ hồ

$R \rightarrow a \mid b \mid c \mid \dots$

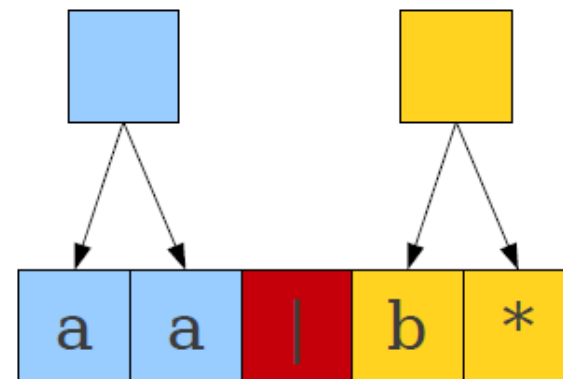
$R \rightarrow \epsilon$

$R \rightarrow RR$

$R \rightarrow R \mid R$

$R \rightarrow R^*$

$R \rightarrow (R)$



# Khử mơ hồ

$R \rightarrow a \mid b \mid c \mid \dots$

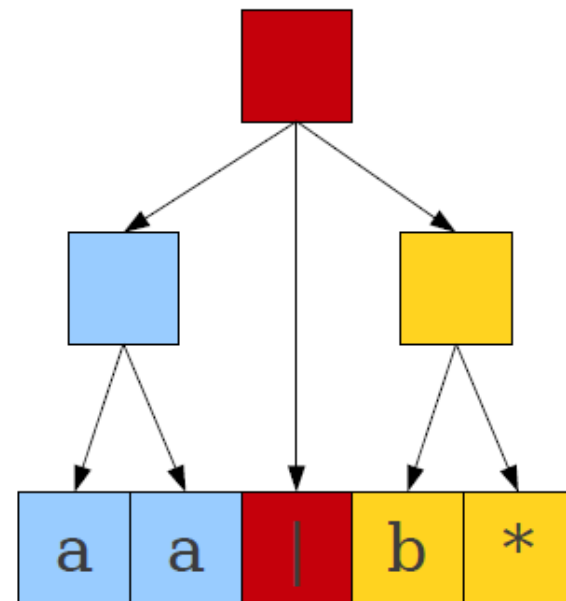
$R \rightarrow \epsilon$

$R \rightarrow RR$

$R \rightarrow R \mid R$

$R \rightarrow R^*$

$R \rightarrow (R)$



# Khử mơ hồ

$$R \rightarrow a \mid b \mid c \mid \dots$$

$$R \rightarrow \epsilon$$

$$R \rightarrow RR$$

$$R \rightarrow R \mid R$$

$$R \rightarrow R^*$$

$$R \rightarrow (R)$$

$$R \rightarrow S \mid R \mid S$$

$$S \rightarrow T \mid ST$$

$$T \rightarrow U \mid T^*$$

$$U \rightarrow a \mid b \mid c \mid \dots$$

$$U \rightarrow \epsilon$$

$$U \rightarrow (R)$$

# Tại sao văn phạm không mơ hồ ?

$$R \rightarrow S \mid R \text{ “|” } S$$
$$S \rightarrow T \mid ST$$
$$T \rightarrow U \mid T^*$$
$$U \rightarrow a \mid b \mid \dots$$
$$U \rightarrow \text{“}\epsilon\text{”}$$
$$U \rightarrow (R)$$

# Tại sao văn phạm không mơ hồ ?

$R \rightarrow S \mid R \mid S$

$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$

Chỉ sinh ra các biểu thức  
“nguyên tố”



# Tại sao văn phạm không mơ hồ ?

$R \rightarrow S \mid R \mid S$

$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

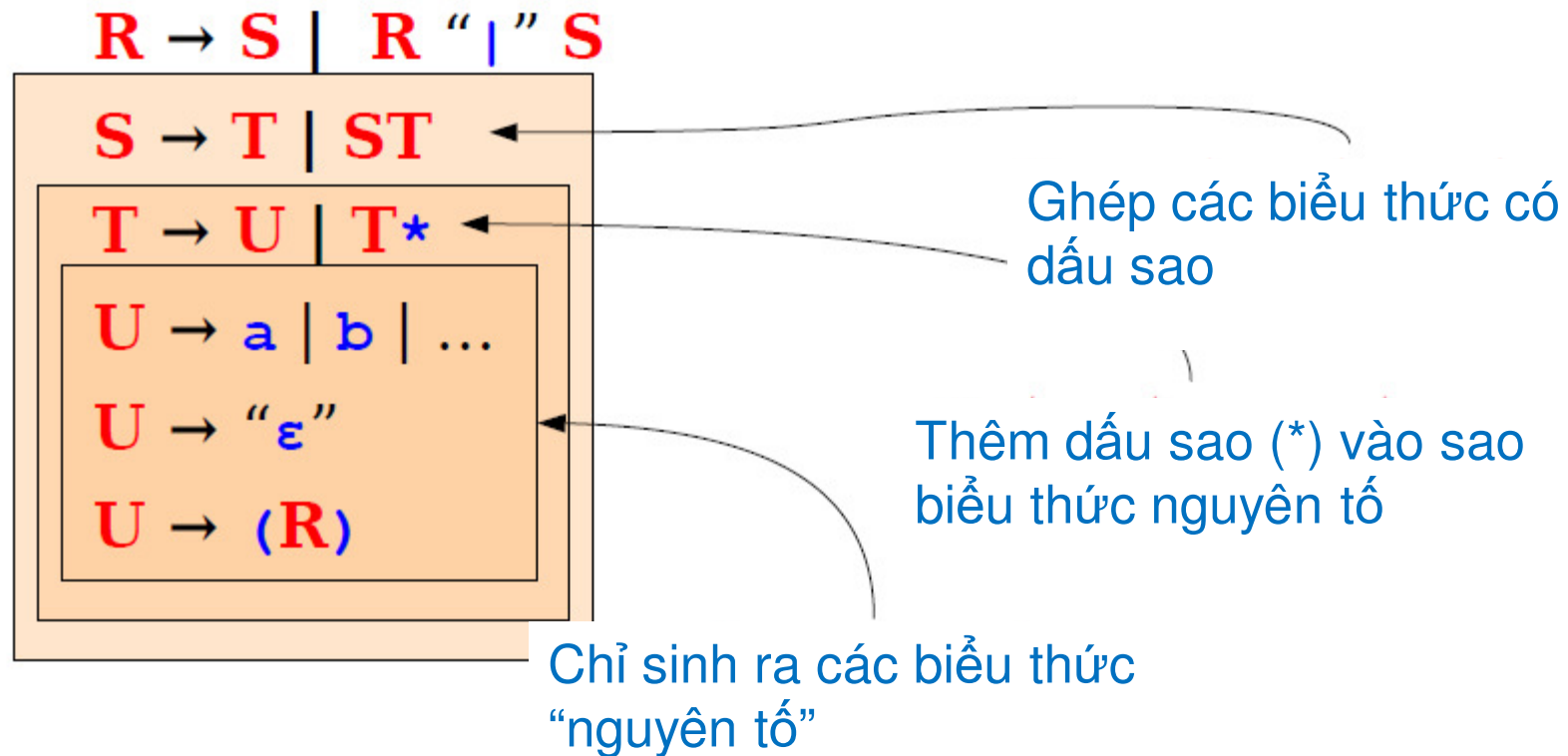
$U \rightarrow \epsilon$

$U \rightarrow (R)$

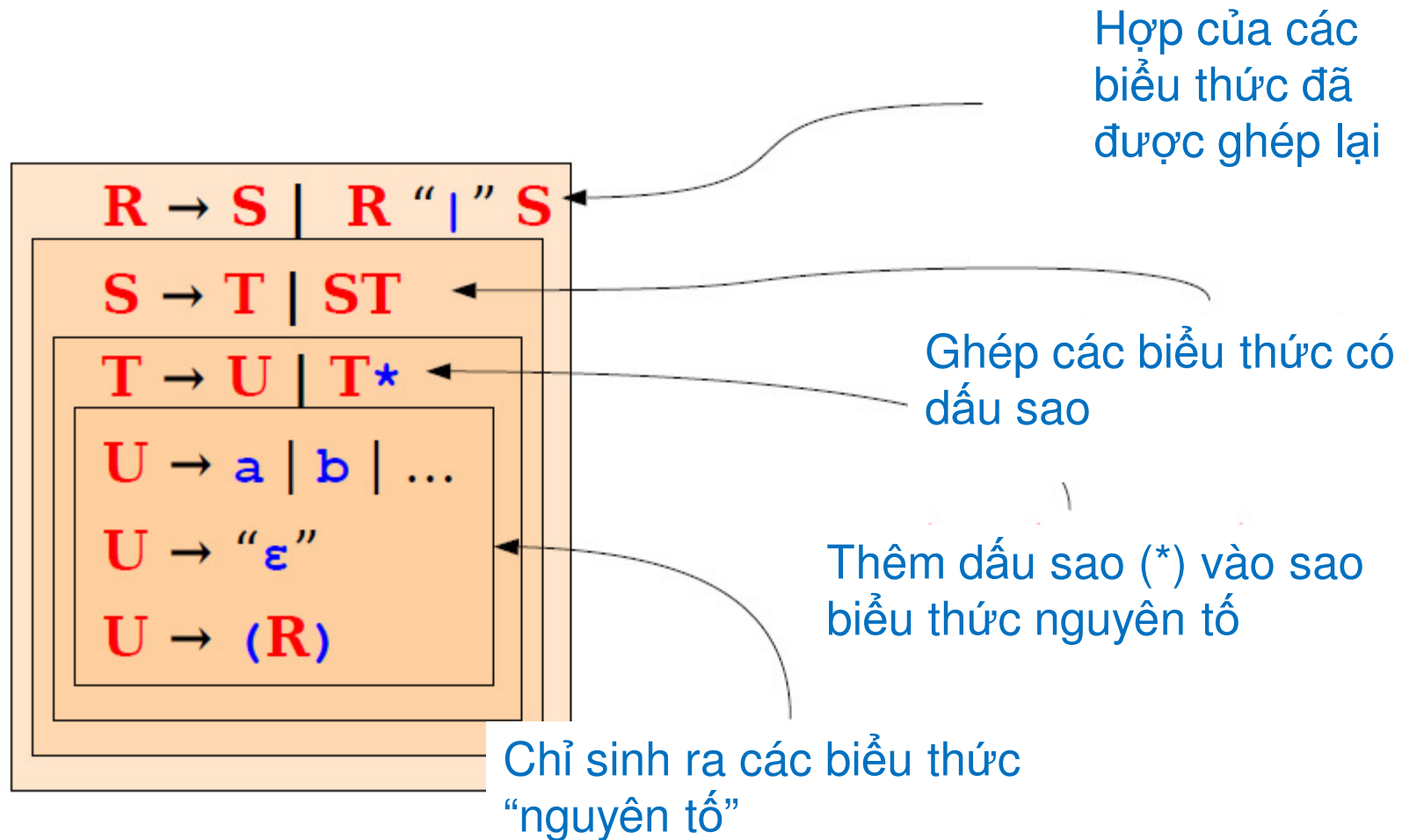
Thêm dấu sao (\*) vào sau biểu thức nguyên tố

Chỉ sinh ra các biểu thức “nguyên tố”

# Tại sao văn phạm không mơ hồ ?



# Tại sao văn phạm không mơ hồ ?



R

$R \rightarrow S \mid R \mid S$

$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$

|   |   |  |   |  |   |   |
|---|---|--|---|--|---|---|
| a | b |  | c |  | a | * |
|---|---|--|---|--|---|---|

$R \rightarrow S \mid R \mid S$

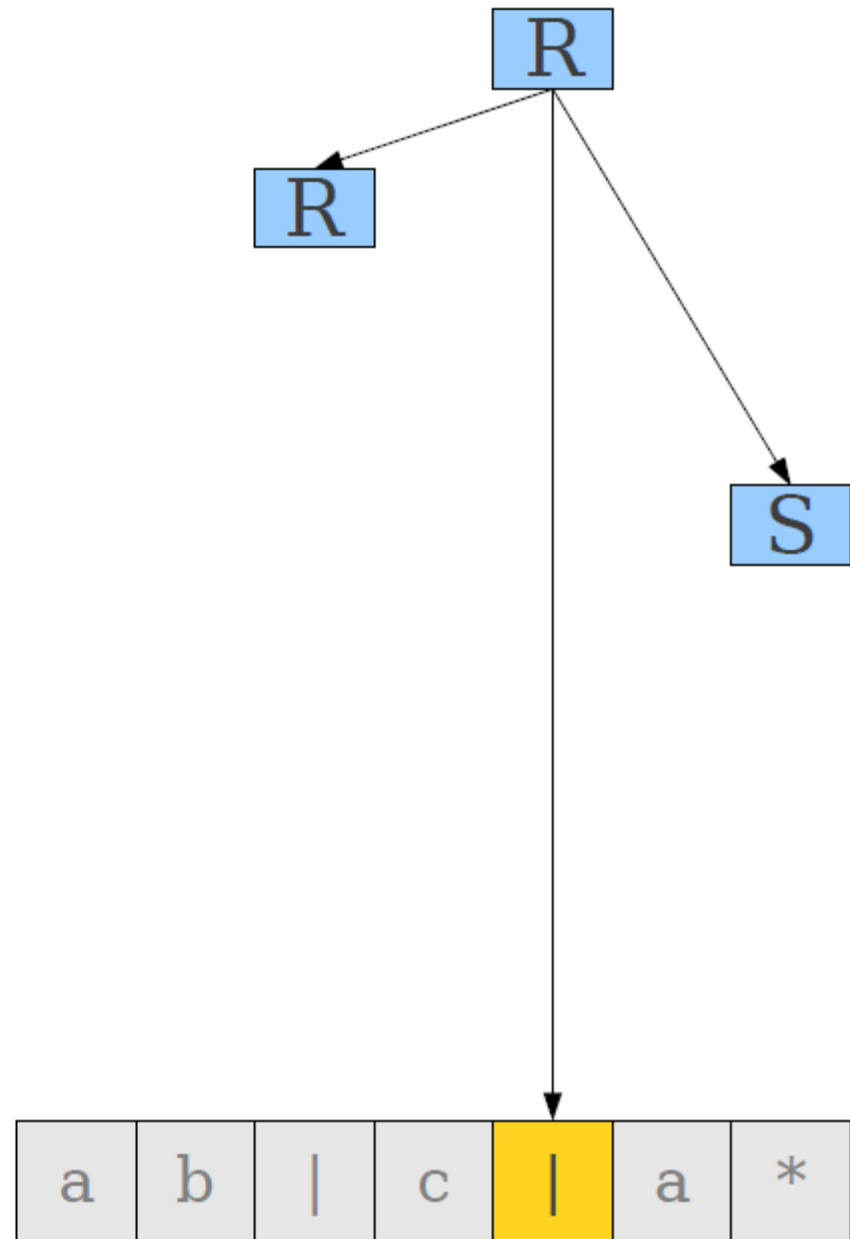
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$

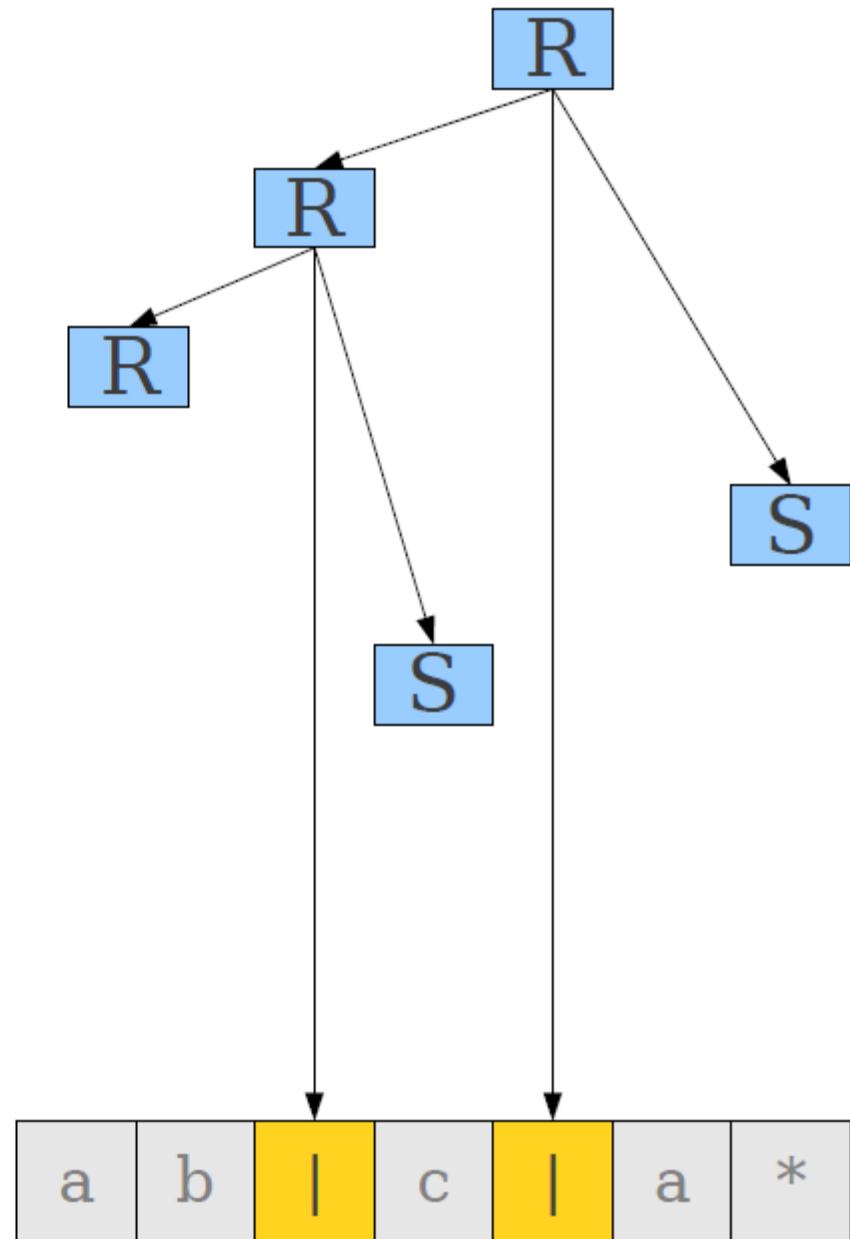


$$R \rightarrow S \mid R \text{ " | " } S$$

**S**  $\rightarrow$  **T** | **ST**

$$T \rightarrow U \mid T^*$$
$$\mathbf{U} \rightarrow \mathbf{a} \mid \mathbf{b} \mid \mathbf{c} \mid \dots$$

**U**  $\rightarrow$  “ **$\epsilon$** ”

$$\mathbf{U} \rightarrow (\mathbf{R})$$


**R**  $\rightarrow$  **S** | **R** " | " **S**

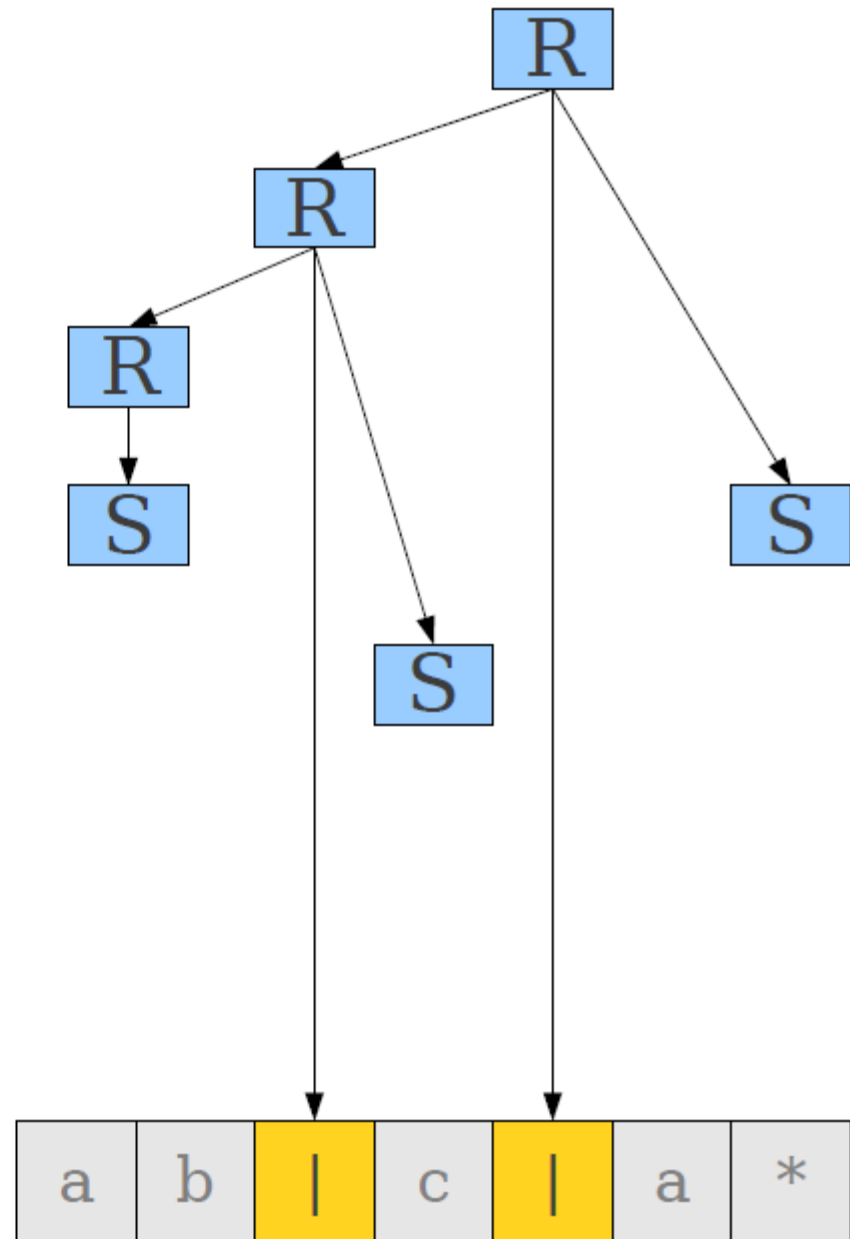
**S**  $\rightarrow$  **T** | **ST**

$$T \rightarrow U \mid T^*$$

**U**  $\rightarrow$  a | b | c | ...

**U**  $\rightarrow$  "ε"

**U** → **(R)**



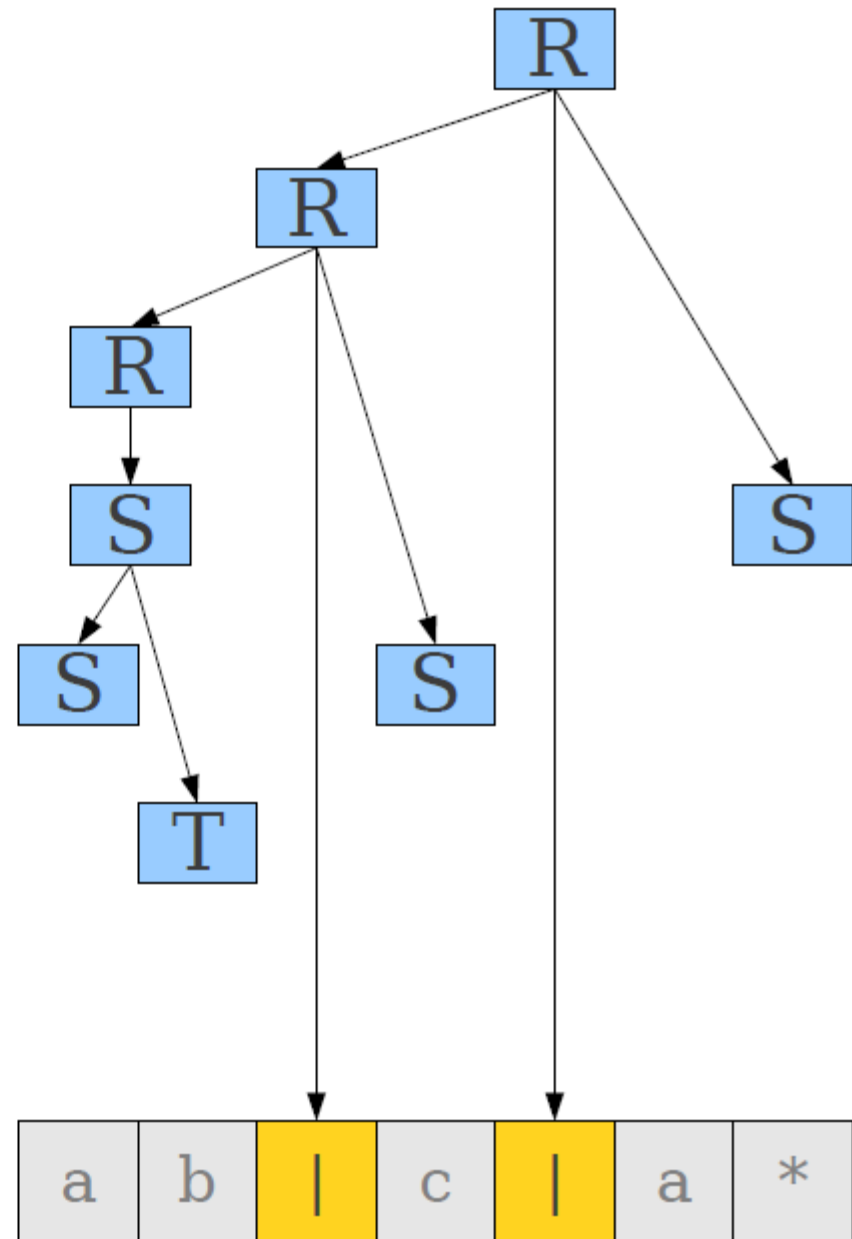
**R**  $\rightarrow$  **S** | **R** " | " **S**

**S**  $\rightarrow$  **T** | **ST**

$$T \rightarrow U \mid T^*$$

$\mathbf{U} \rightarrow a \mid b \mid c \mid \dots$

**U**  $\rightarrow$  “ **$\varepsilon$** ”

$$U \rightarrow (R)$$




**R**  $\rightarrow$  **S** | **R** " | " **S**

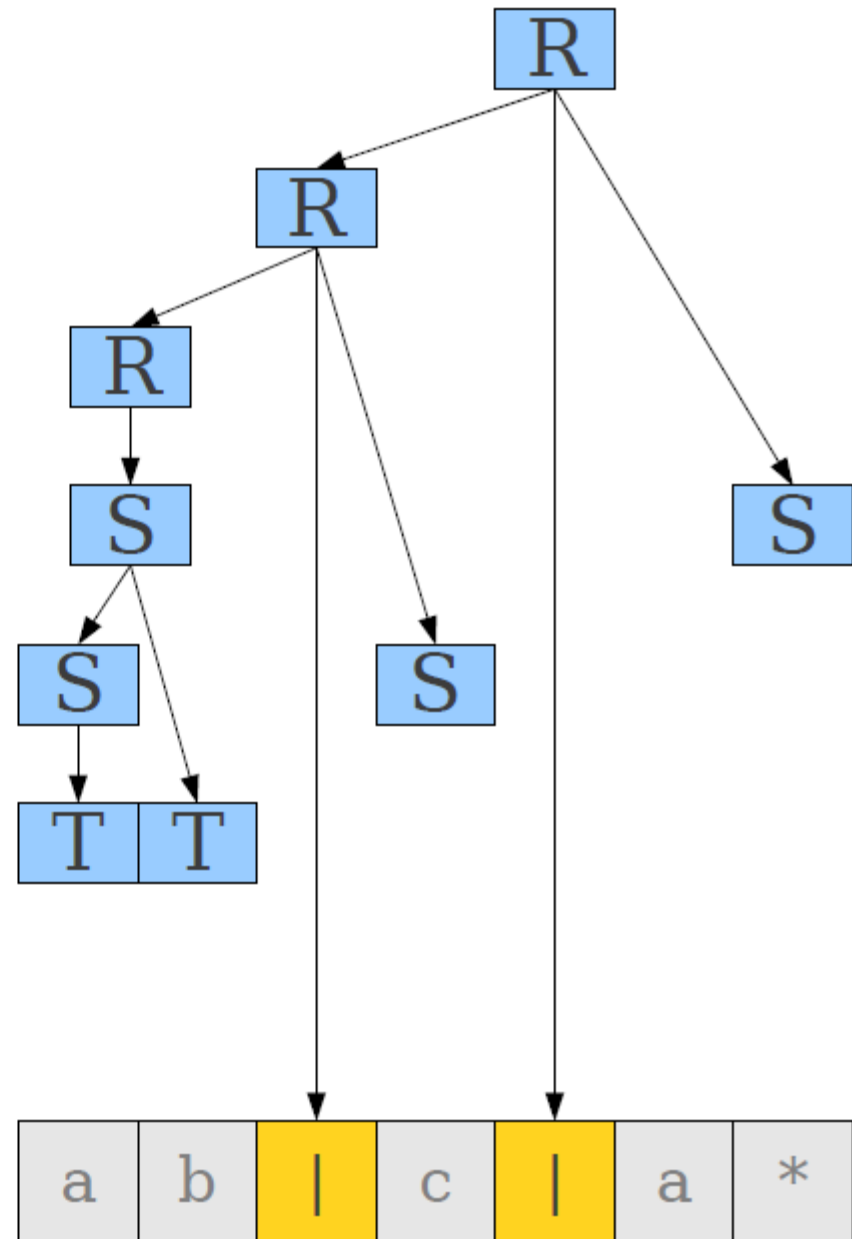
**S**  $\rightarrow$  **T** | **ST**

$$T \rightarrow U \mid T^*$$

$\mathbf{U} \rightarrow a \mid b \mid c \mid \dots$

**U**  $\rightarrow$  “ **$\varepsilon$** ”

**U**  $\rightarrow$  **(R)**



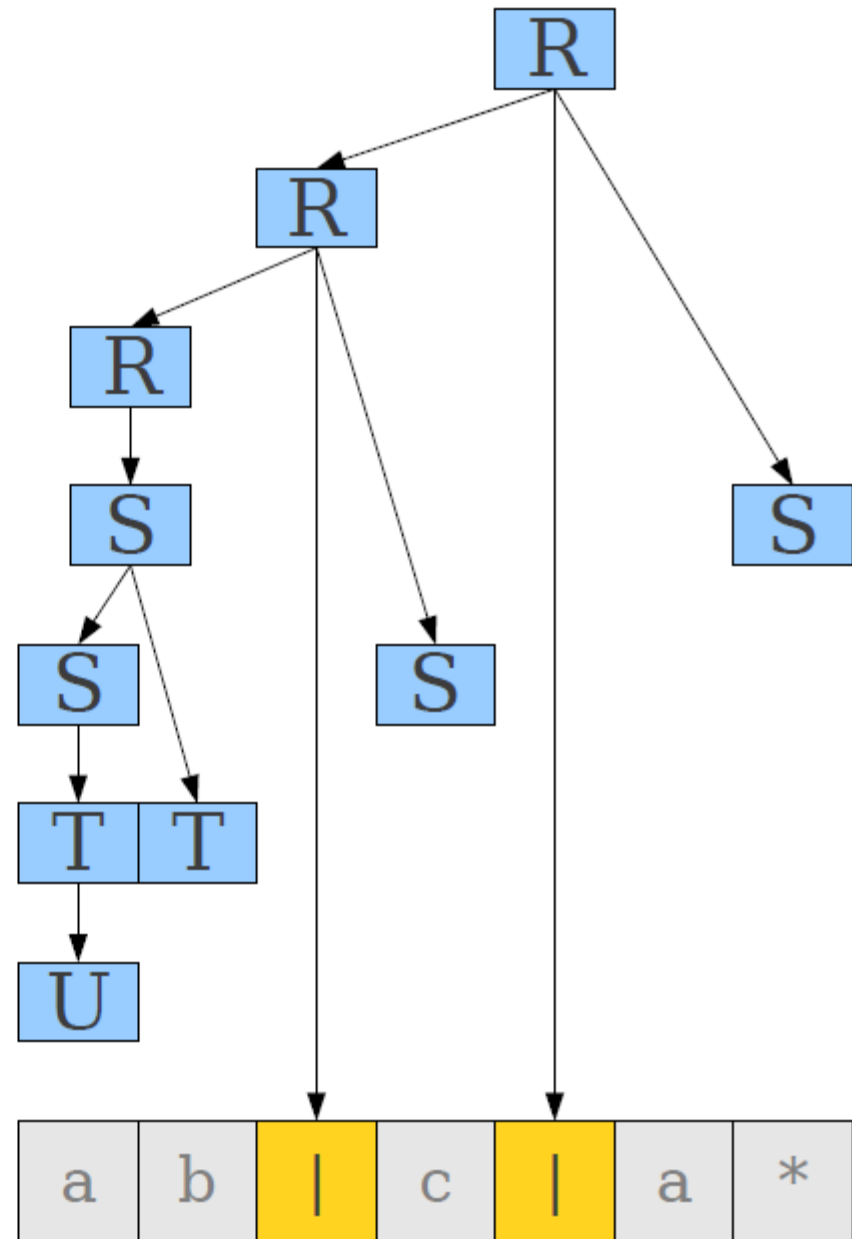
**R**  $\rightarrow$  **S** | **R** " | " **S**

**S**  $\rightarrow$  **T** | **ST**

$$T \rightarrow U \mid T^*$$

$\mathbf{U} \rightarrow a \mid b \mid c \mid \dots$

**U**  $\rightarrow$  “ **$\varepsilon$** ”

$$U \rightarrow (R)$$


$R \rightarrow S \mid R \mid S$

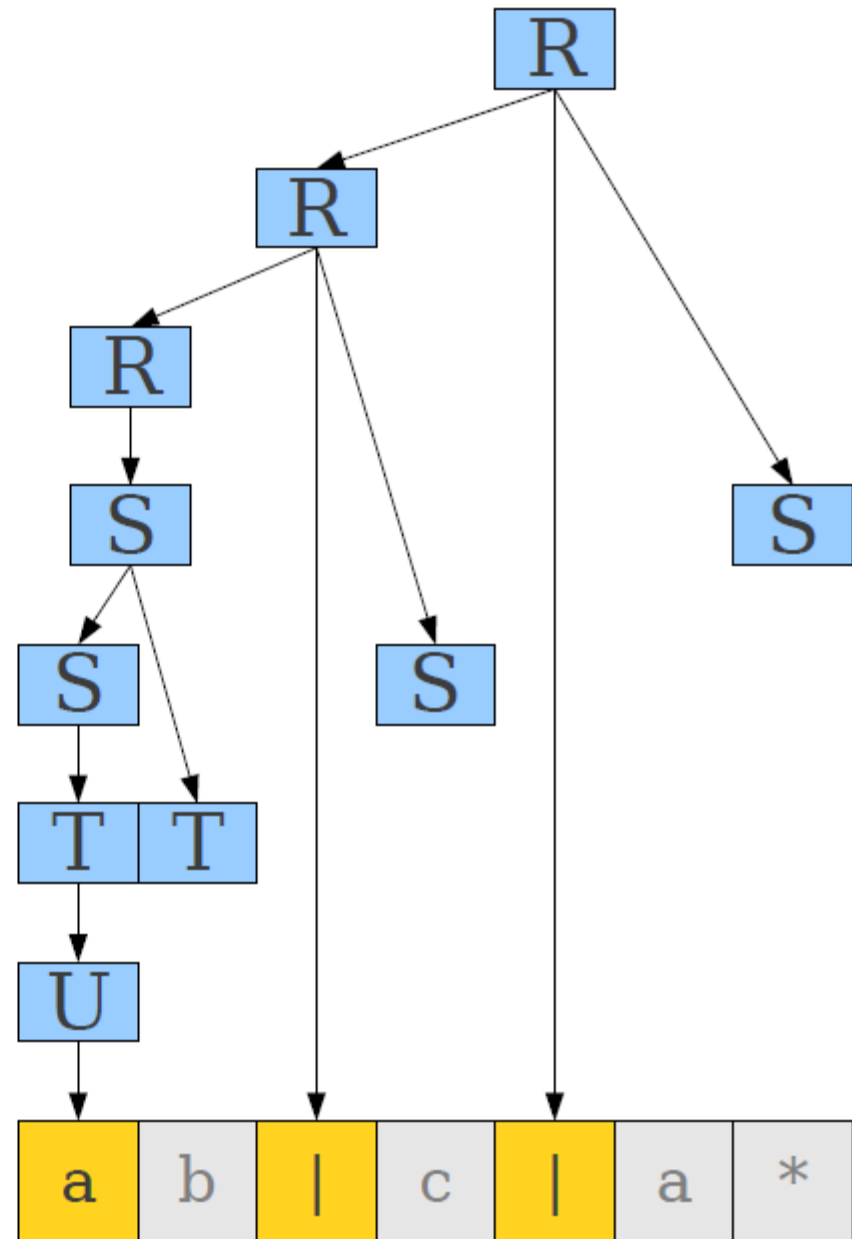
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

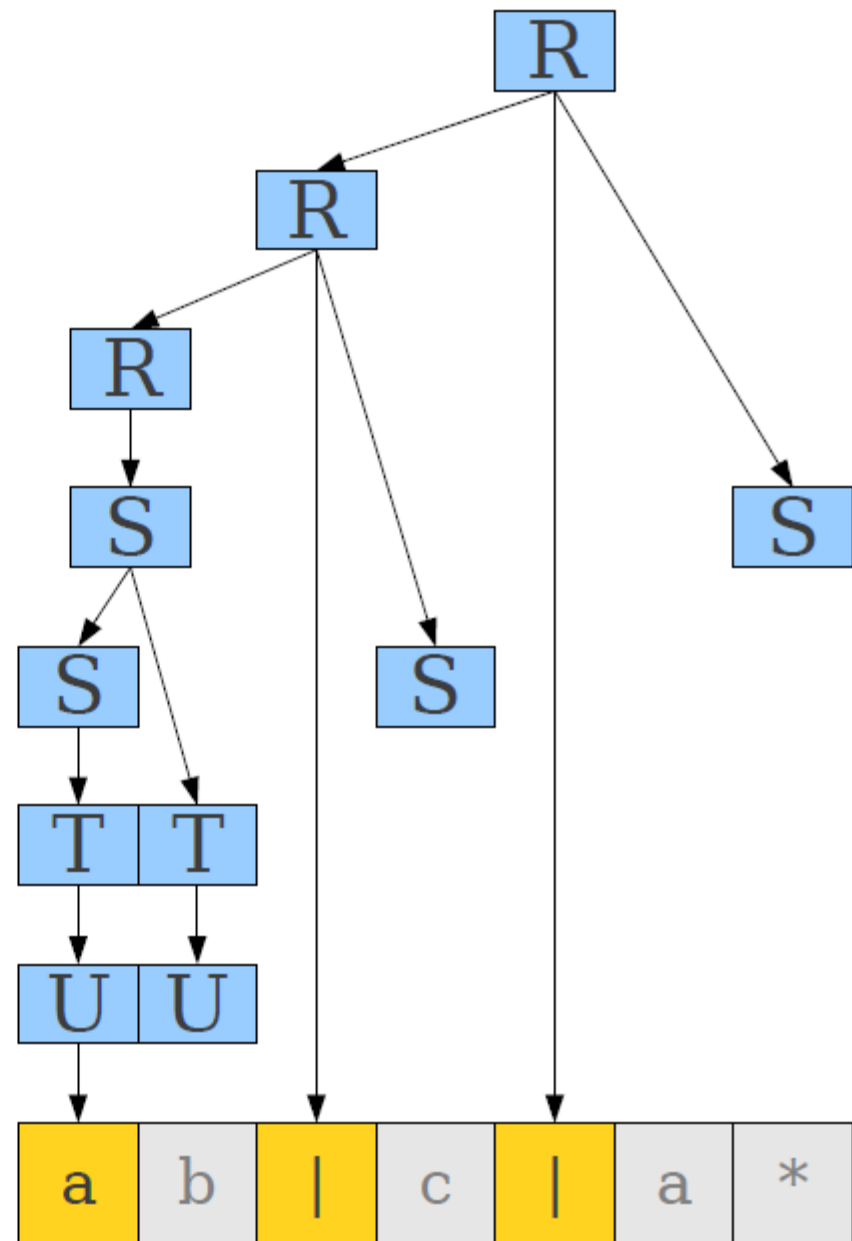
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

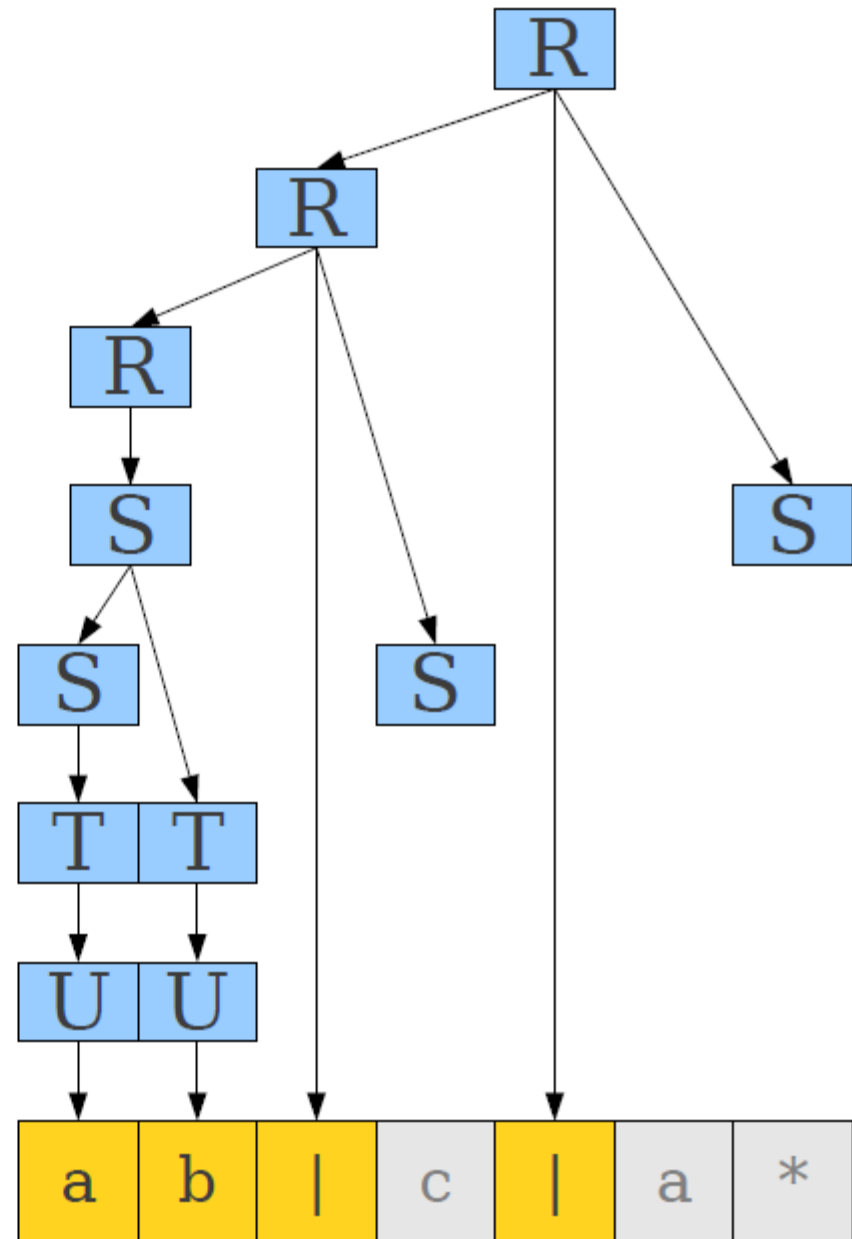
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

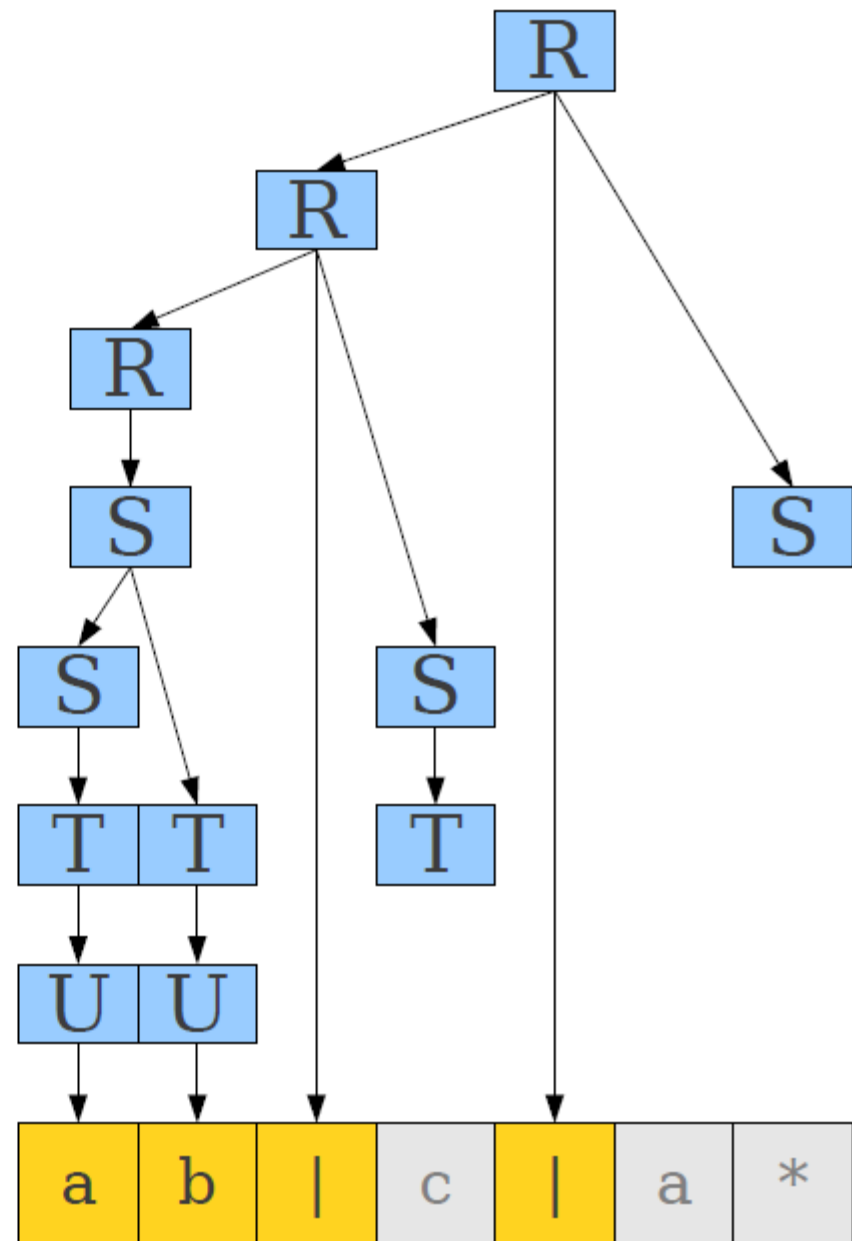
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

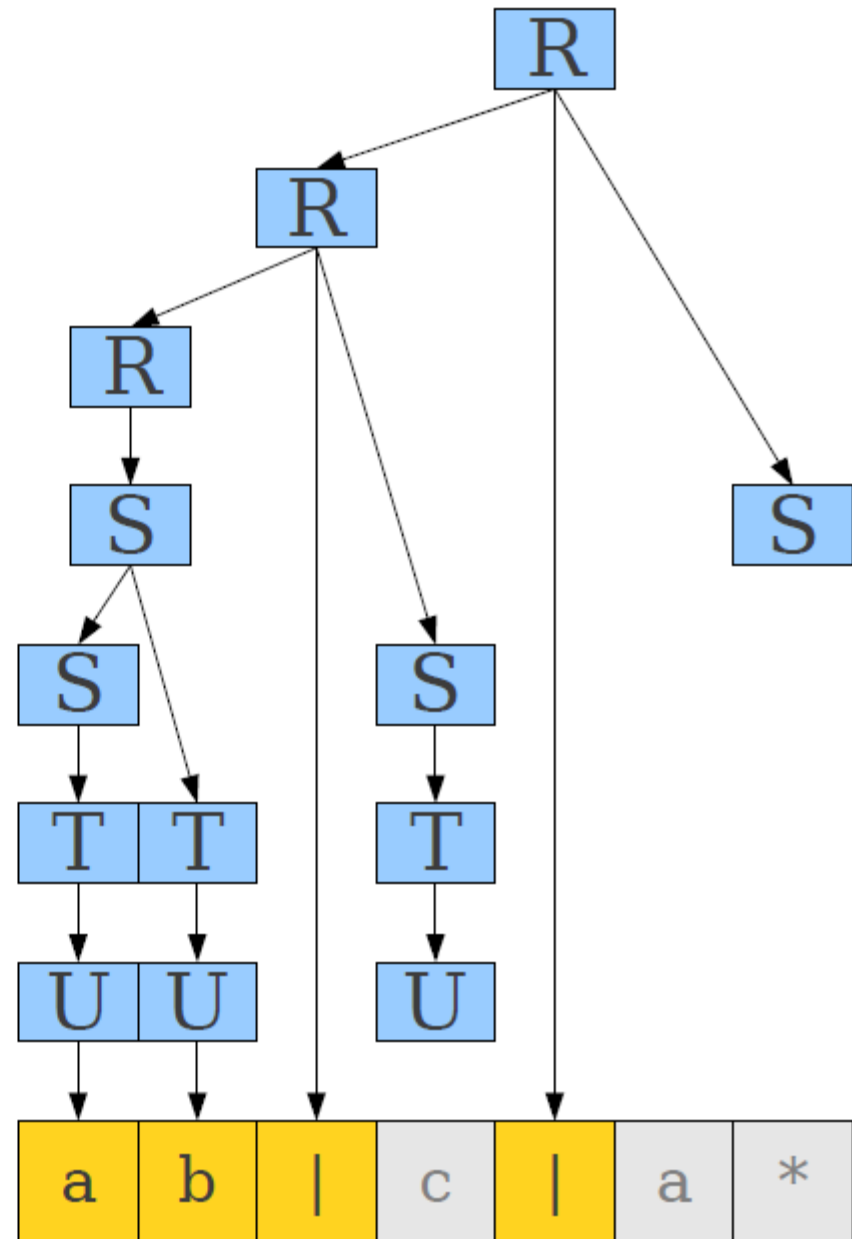
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

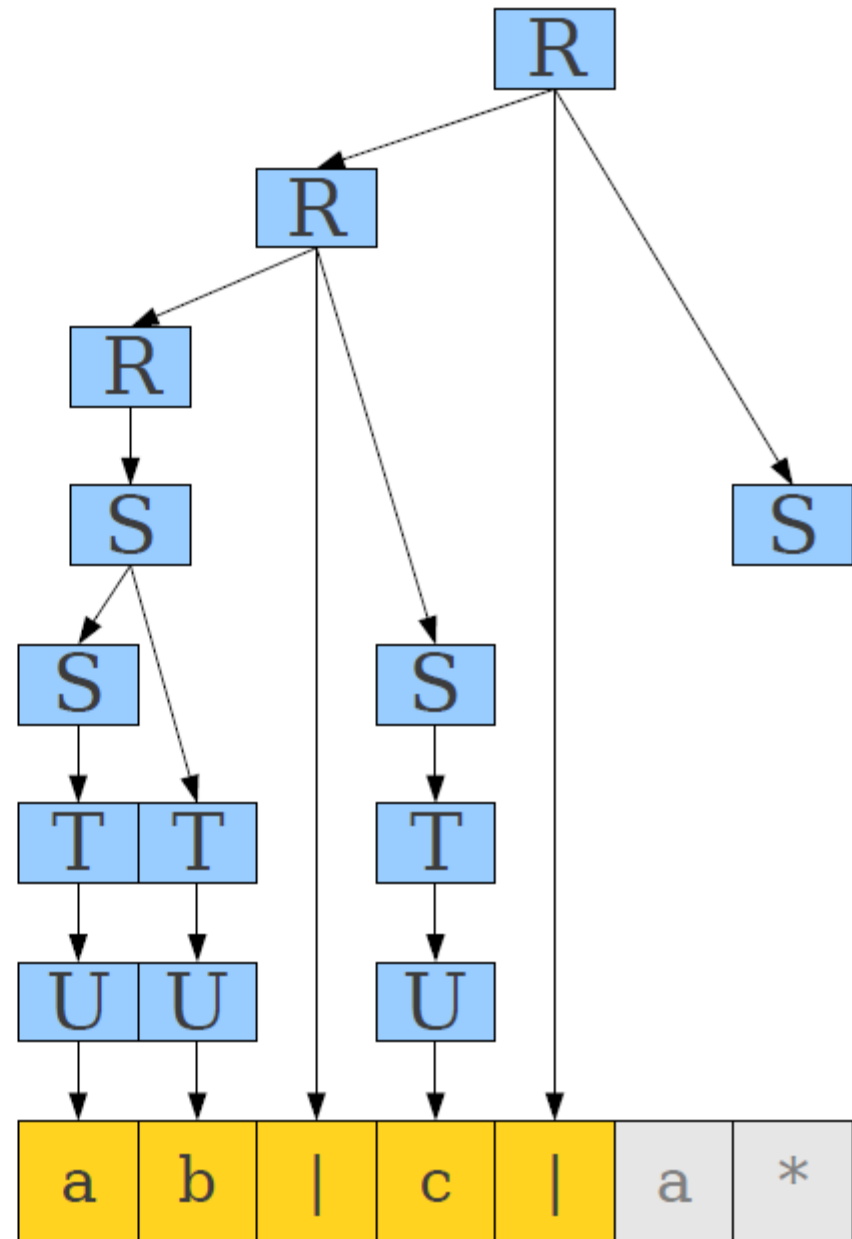
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$





**R**  $\rightarrow$  **S** | **R** " | " **S**

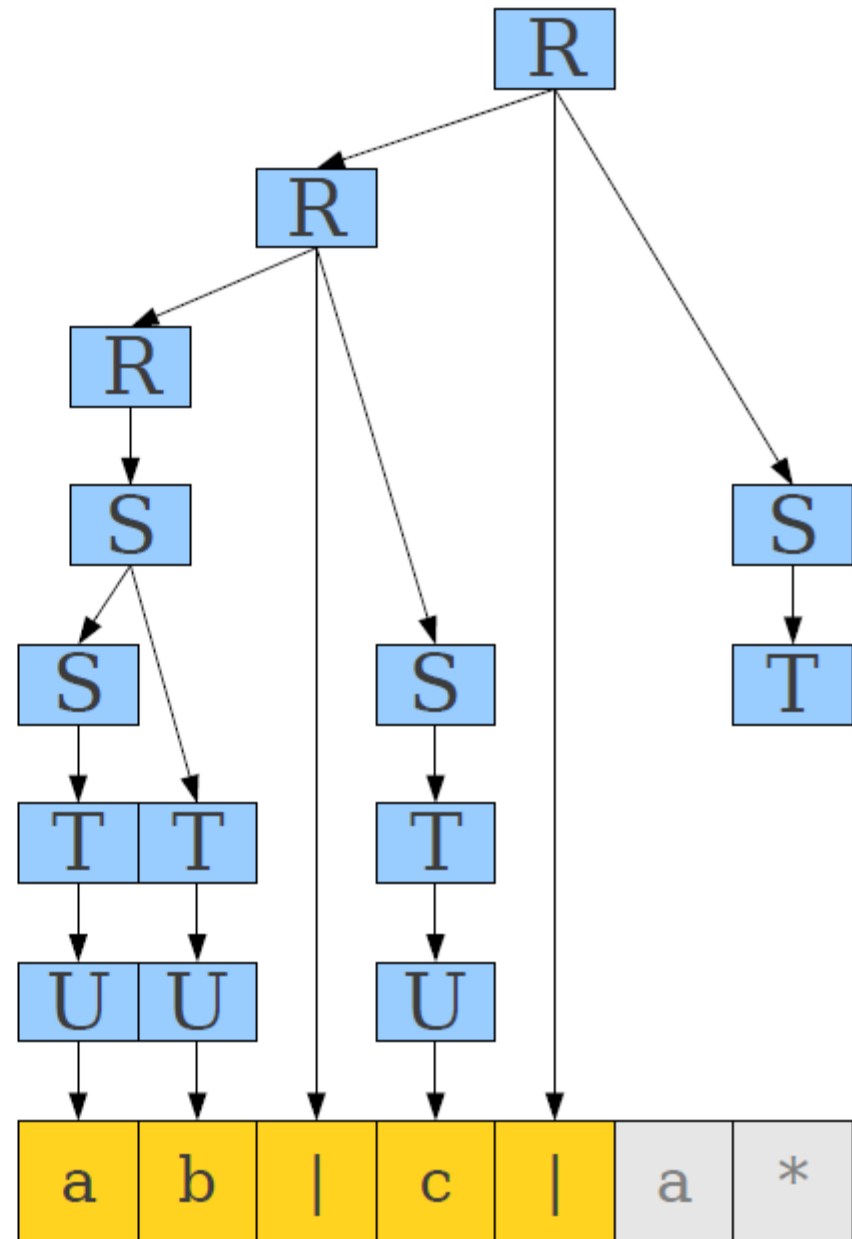
**S**  $\rightarrow$  **T** | **ST**

$$T \rightarrow U \mid T^*$$

$\mathbf{U} \rightarrow a \mid b \mid c \mid \dots$

**U**  $\rightarrow$  “ **$\varepsilon$** ”

**U**  $\rightarrow$  **(R)**



$R \rightarrow S \mid R \mid S$

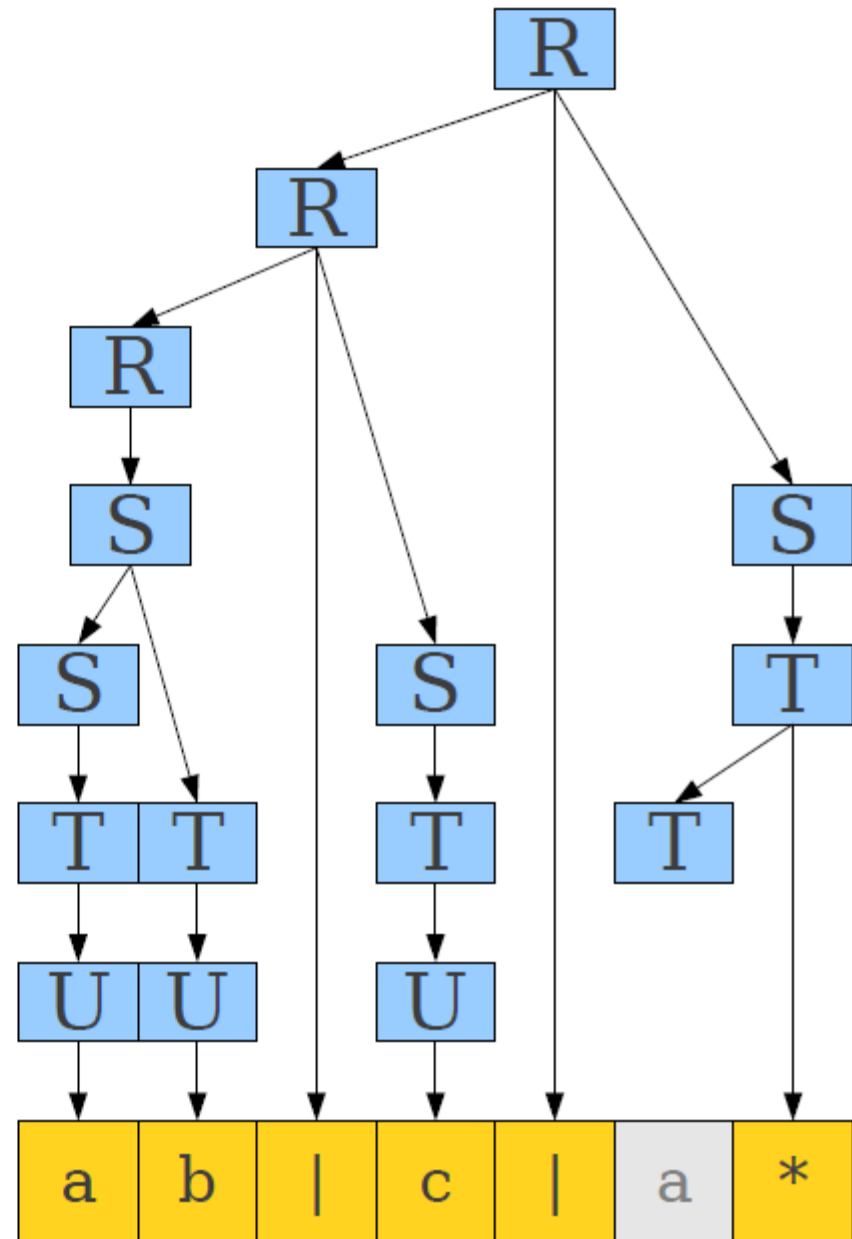
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

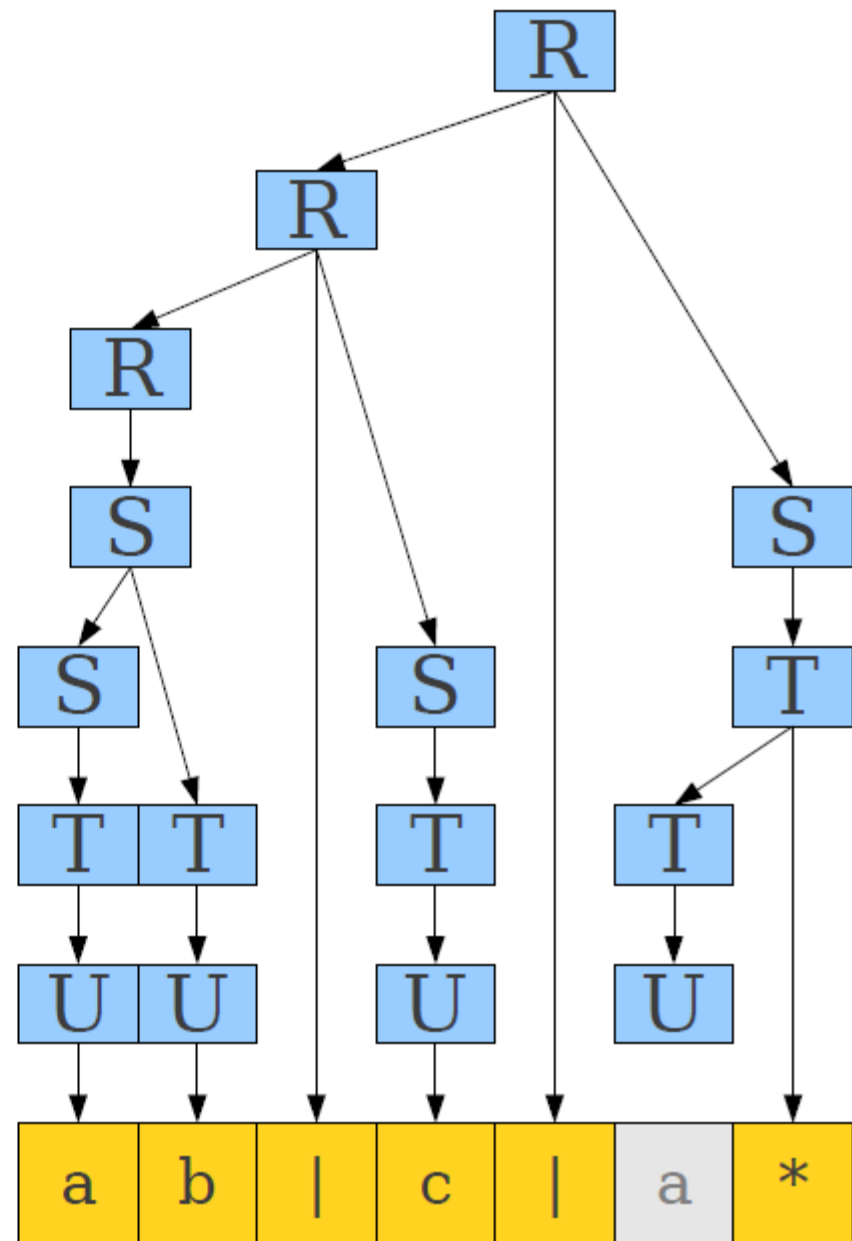
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

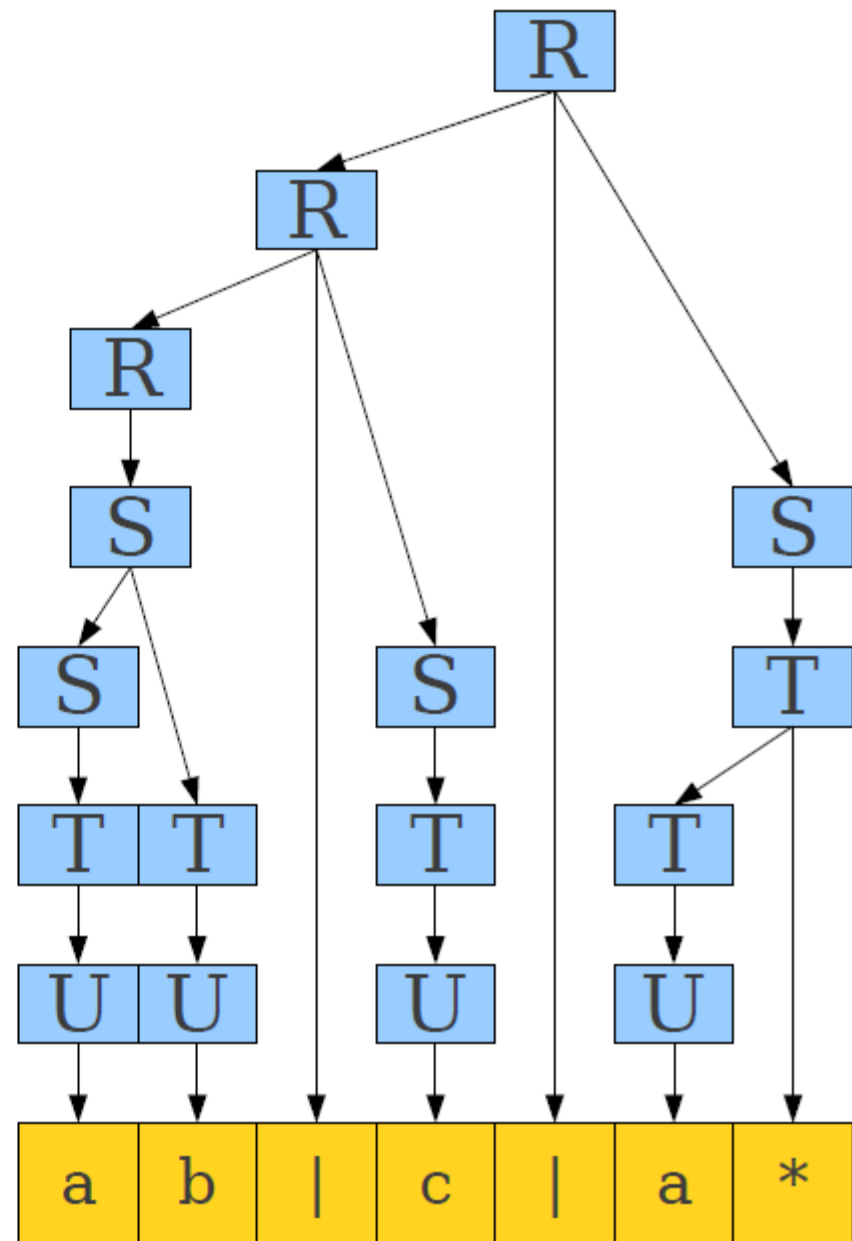
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid c \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



# Khai báo độ ưu tiên

- Nếu ta không phân tầng văn phạm, sự mơ hồ của văn phạm cũng có thể được khử bằng cách **khai báo độ ưu tiên**.
  - Ví dụ: phép toán nhân có độ ưu tiên cao hơn phép toán cộng nhưng có độ ưu tiên nhỏ hơn phép lũy thừa.
- Khai báo độ ưu tiên cho phép phân tích cú pháp một văn phạm mơ hồ.
- Ta sẽ thấy điều này được cài đặt như thế nào trong các phần sau.

# Cấu trúc cây phân tích cú pháp

$R \rightarrow S \mid R \text{ “|” } S$

$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \text{“}\epsilon\text{”}$

$U \rightarrow (R)$

# Cấu trúc cây phân tích cú pháp

$R \rightarrow S \mid R \mid S$

$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$

|   |   |  |   |   |
|---|---|--|---|---|
| a | a |  | b | * |
|---|---|--|---|---|

# Cấu trúc cây phân tích cú pháp

$R \rightarrow S \mid R \mid S$

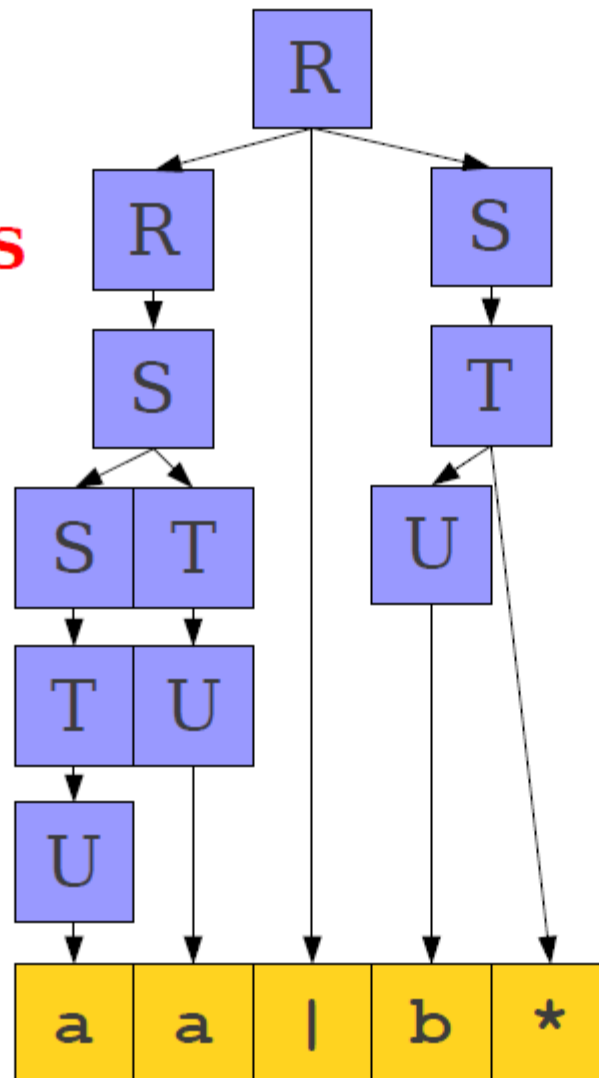
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$





# Cấu trúc cây phân tích cú pháp

$R \rightarrow S \mid R \mid S$

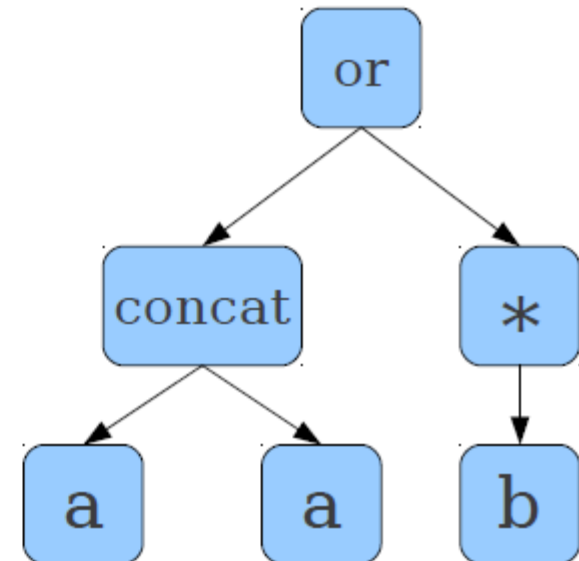
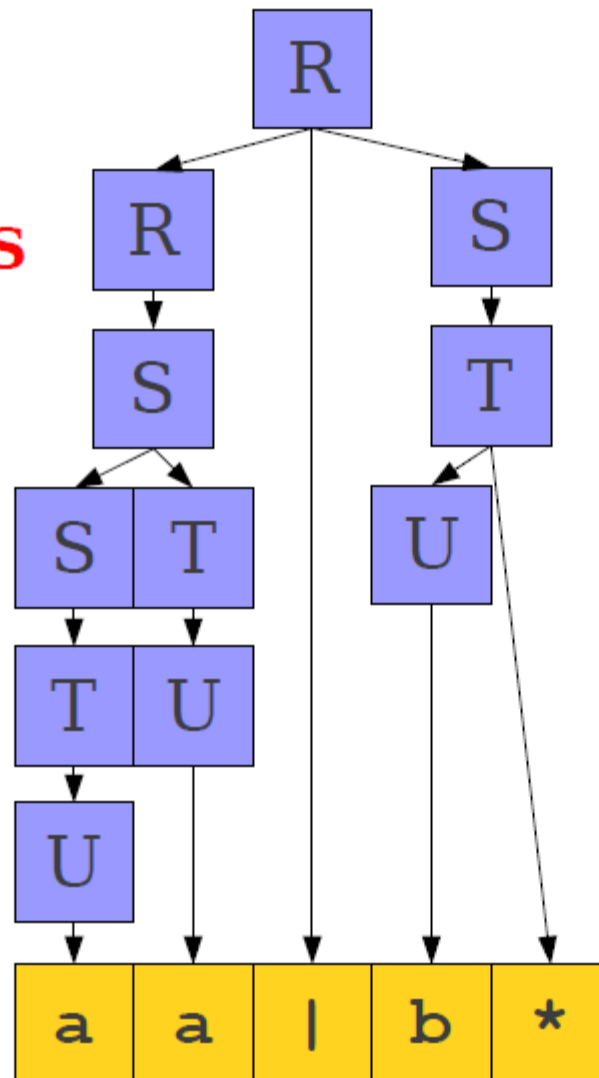
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



# Cấu trúc cây phân tích cú pháp

$R \rightarrow S \mid R \text{ “|” } S$

$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \text{“}\epsilon\text{”}$

$U \rightarrow (R)$

$R \rightarrow S \mid R \mid S$

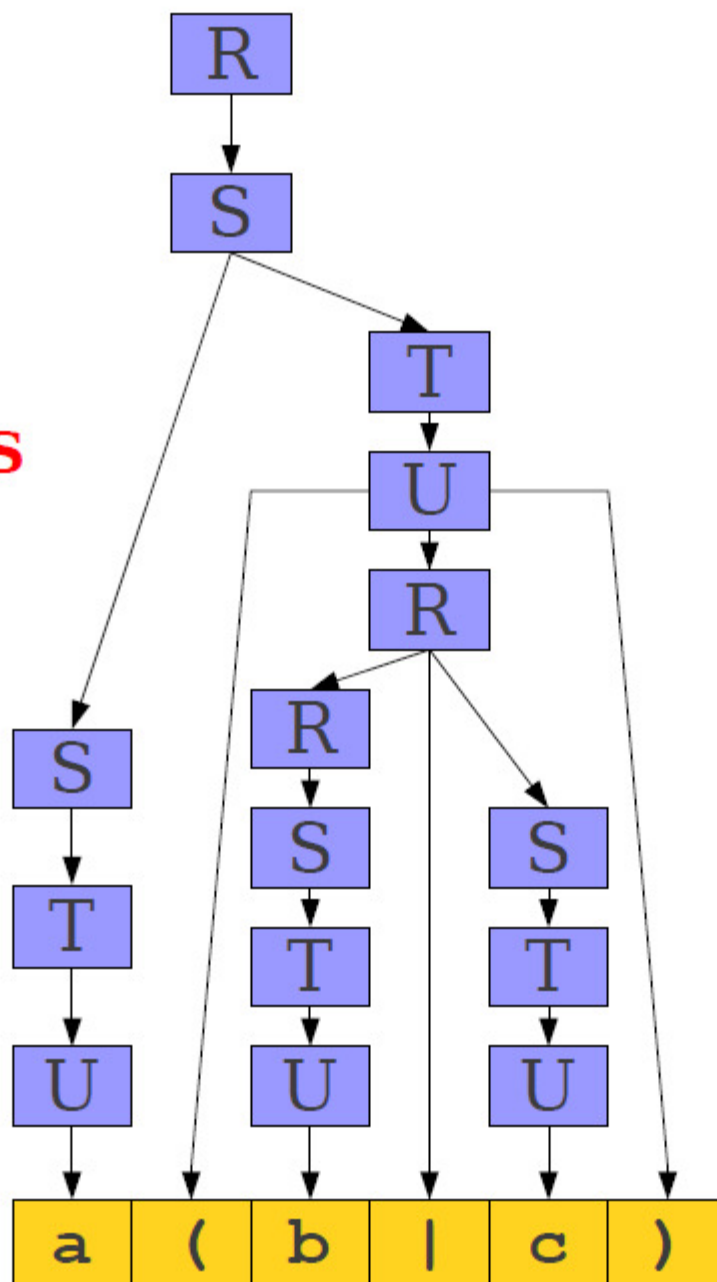
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



$R \rightarrow S \mid R \mid S$

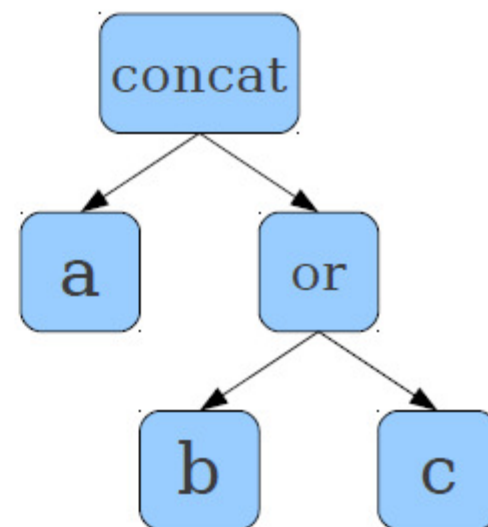
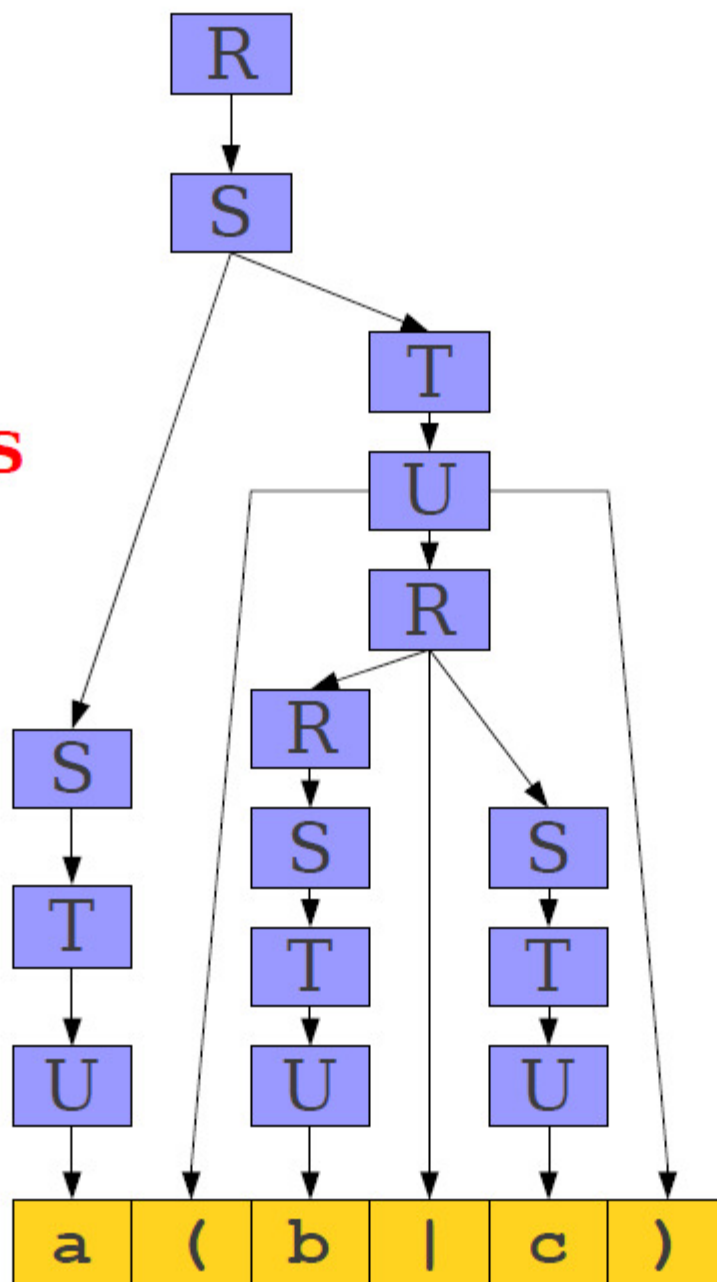
$S \rightarrow T \mid ST$

$T \rightarrow U \mid T^*$

$U \rightarrow a \mid b \mid \dots$

$U \rightarrow \epsilon$

$U \rightarrow (R)$



# Câu cú pháp trừu tượng

- Câu cú pháp trừu tượng (Abstract Syntax Trees, gọi tắt là AST)
  - Cây phân tích cú pháp là một **cây cú pháp cụ thể (concrete syntax tree)**; mô tả cách dẫn xuất ra chuỗi đầu vào
- Một cấu trúc hữu dụng hơn là **cây cú pháp trừu tượng (abstract syntax tree)**, được đơn giản từ cây cú pháp cụ thể, chỉ giữ lại cấu trúc chính của đầu vào.

# Xây dựng cây cú pháp trừu tượng

- Cây cú pháp trừu tượng thường được xây dựng thông qua các **hành vi ngữ nghĩa (semantic actions)** của luật sinh.
- Hành vi ngữ nghĩa là một phần mã lệnh thực thi được gắn kết với mỗi luật sinh.
- Khi phân tích đầu vào, mã lệnh này sẽ được thực thi để xây dựng cây cú pháp trừu tượng.
- Thứ tự thực thi của các đoạn mã lệnh phụ thuộc vào phương pháp phân tích cú pháp được sử dụng.
- Đây được gọi là **dịch trực tiếp cú pháp (syntax-directed translation)**. Xem giáo trình !

# Hành vi ngữ nghĩa đơn giản

$E \rightarrow T + E$        $E_1.val = T.val + E_2.val$

$E \rightarrow T$        $E.val = T.val$

$T \rightarrow int$        $T.val = int.val$

$T \rightarrow int * T$        $T.val = int.val * T.val$

$T \rightarrow (E)$        $T.val = E.val$

# Hành vi ngữ nghĩa đơn giản

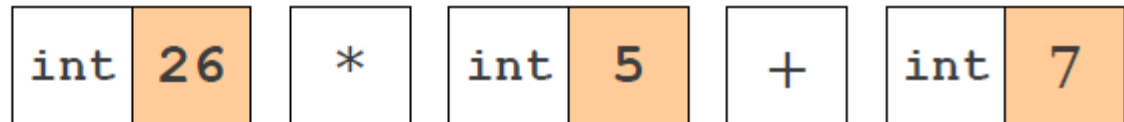
$E \rightarrow T + E$        $E_1.val = T.val + E_2.val$

$E \rightarrow T$        $E.val = T.val$

$T \rightarrow int$        $T.val = int.val$

$T \rightarrow int * T$        $T.val = int.val * T.val$

$T \rightarrow (E)$        $T.val = E.val$





# Hành vi ngữ nghĩa đơn giản

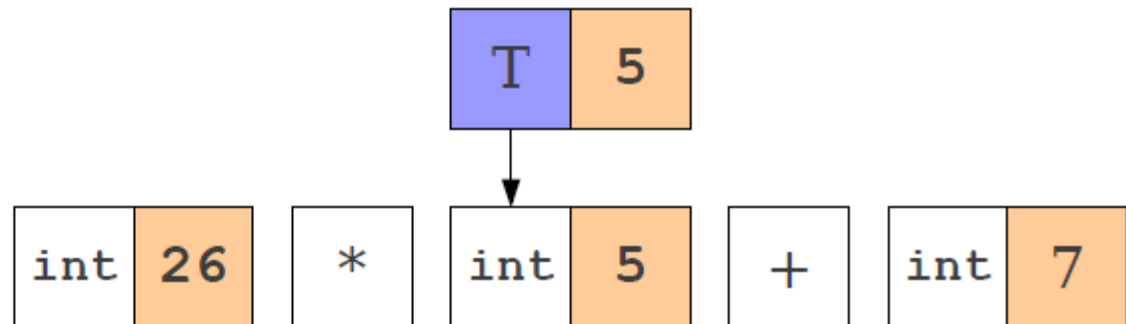
$E \rightarrow T + E$        $E_1.val = T.val + E_2.val$

$E \rightarrow T$        $E.val = T.val$

$T \rightarrow int$        $T.val = int.val$

$T \rightarrow int * T$        $T.val = int.val * T.val$

$T \rightarrow (E)$        $T.val = E.val$



# Hành vi ngữ nghĩa đơn giản

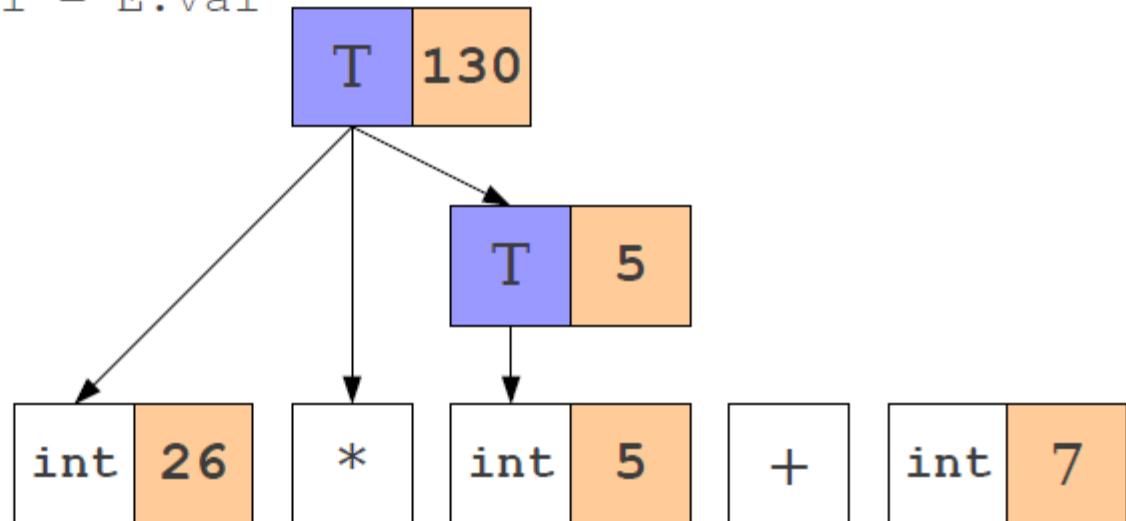
$E \rightarrow T + E$        $E_1.val = T.val + E_2.val$

$E \rightarrow T$        $E.val = T.val$

$T \rightarrow int$        $T.val = int.val$

$T \rightarrow int * T$        $T.val = int.val * T.val$

$T \rightarrow (E)$        $T.val = E.val$



# Hành vi ngữ nghĩa đơn giản

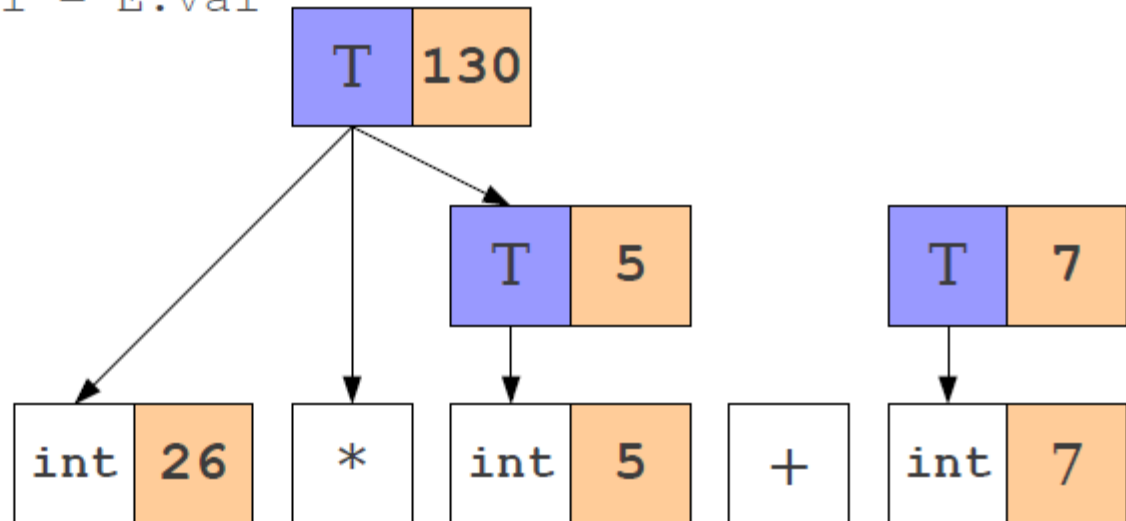
$E \rightarrow T + E$        $E_1.val = T.val + E_2.val$

$E \rightarrow T$        $E.val = T.val$

$T \rightarrow int$        $T.val = int.val$

$T \rightarrow int * T$        $T.val = int.val * T.val$

$T \rightarrow (E)$        $T.val = E.val$



# Hành vi ngữ nghĩa đơn giản

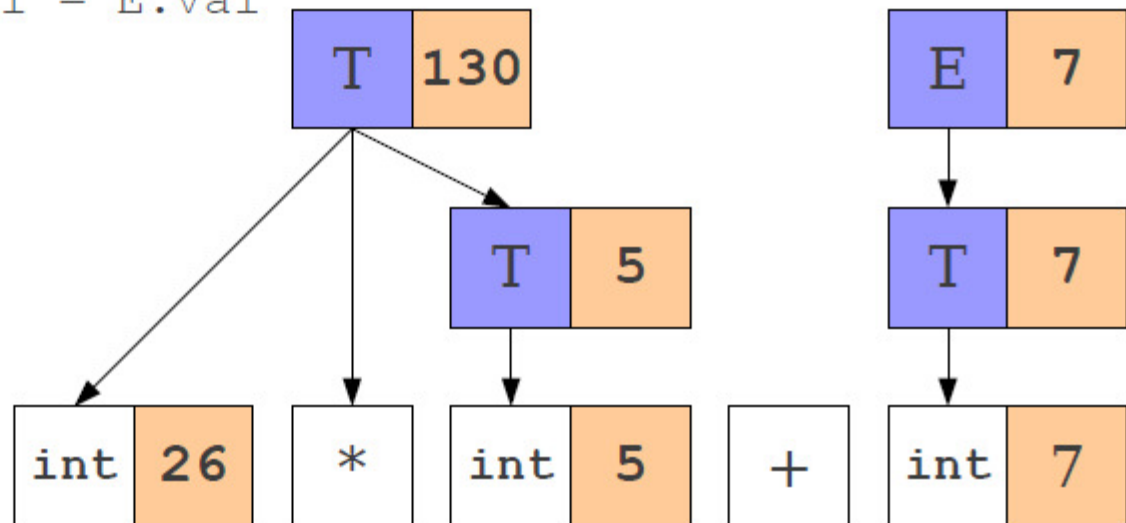
$E \rightarrow T + E$        $E_1.val = T.val + E_2.val$

$E \rightarrow T$        $E.val = T.val$

$T \rightarrow int$        $T.val = int.val$

$T \rightarrow int * T$        $T.val = int.val * T.val$

$T \rightarrow (E)$        $T.val = E.val$



# Hành vi ngữ nghĩa đơn giản

$E \rightarrow T + E$

$E_1.val = T.val + E_2.val$

$E \rightarrow T$

$E.val = T.val$

$T \rightarrow int$

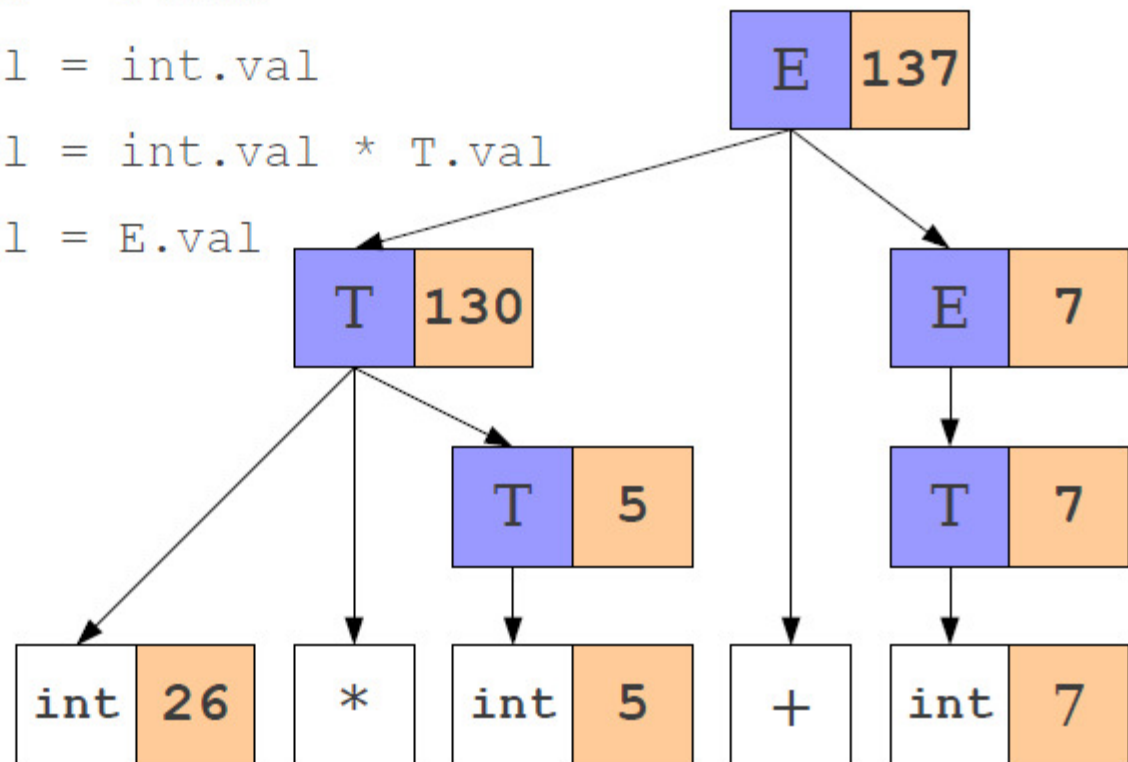
$T.val = int.val$

$T \rightarrow int * T$

$T.val = int.val * T.val$

$T \rightarrow (E)$

$T.val = E.val$



# Hành vi ngữ nghĩa xây dựng AST

|                          |                                                                        |
|--------------------------|------------------------------------------------------------------------|
| $R \rightarrow S$        | <code>R.ast = S.ast;</code>                                            |
| $R \rightarrow R \mid S$ | <code>R<sub>1</sub>.ast = new Or(R<sub>2</sub>.ast, S.ast);</code>     |
| $S \rightarrow T$        | <code>S.ast = T.ast;</code>                                            |
| $S \rightarrow ST$       | <code>S<sub>1</sub>.ast = new Concat(S<sub>2</sub>.ast, T.ast);</code> |
| $T \rightarrow U$        | <code>T.ast = U.ast;</code>                                            |
| $T \rightarrow T^*$      | <code>T<sub>1</sub>.ast = new Star(T<sub>2</sub>.ast);</code>          |
| $U \rightarrow a$        | <code>U.ast = new SingleChar('a');</code>                              |
| $U \rightarrow \epsilon$ | <code>U.ast = new Epsilon();</code>                                    |
| $U \rightarrow (R)$      | <code>U.ast = R.ast;</code>                                            |

# Tổng kết chương

- **Phân tích cú pháp** (parsing) xây dựng cấu trúc từ các tokens được sinh ra từ bộ phân tích từ vựng (scanner).
- Ngôn ngữ thường được định nghĩa bằng **văn phạm phi ngữ cảnh** (context-free grammars)
- **Cây phân tích cú pháp** mô tả cách thức dẫn xuất (sinh ra) chuỗi đầu vào từ văn phạm.

# Tổng kết chương

- Một văn phạm gọi là **mơ hồ** nếu có nhiều hơn một cây phân tích cú pháp cho cùng một chuỗi đầu vào.
- Không có giải thuật tự động khử mơ hồ, phải thực hiện thủ công.
- **Cây cú pháp trừu tượng** chứa cách biểu diễn trừu tượng của cú pháp chương trình.
- **Hành vi ngữ nghĩa** đi kèm với các luật sinh, có thể được sử dụng để sinh cây cú pháp trừu tượng



# Tuần tiếp theo

- Phân tích cú pháp từ trên xuống
  - Phân tích cú pháp như bài toán tìm kiếm
  - Bộ phân tích cú pháp quay lui
  - Bộ phân tích cú pháp dự đoán
  - LL(1)